

ABSTRACT

This project aims to develop a predictive model for student academic performance using various machine learning techniques. The dataset includes demographic, socio-economic, and behavioral factors such as gender, part-time job status, internet quality, diet quality, family income, parental education, and study environment.

We began with comprehensive data preprocessing including handling missing values, encoding categorical variables, and feature scaling. Exploratory data analysis (EDA) helped us uncover meaningful patterns and relationships. Models such as Multiple Linear Regression, Lasso, Ridge, Support Vector Machines, and Random Forest were applied and compared based on performance metrics.

Assumption checks were conducted for linear models, and feature importance was analyzed for interpretability. The results demonstrate that certain behavioral and environmental factors significantly influence student performance. The findings of this project can support educational institutions in identifying at-risk students and improving academic outcomes through targeted interventions.

CHAPTER 2:

DATA COLLECTION

The dataset used in this study was sourced from [Kaggle](#). It is a synthetically generated dataset that simulates the academic and lifestyle behaviors of 80,000 students.

It includes diverse features such as:

- Study habits
- Mental health
- Family background
- Motivation
- Environmental factors

1.1 Collection Methodology

The dataset was synthetically generated using Python with realistic statistical modeling, Gaussian distributions, conditional logic, and heuristics to simulate actual student behavior and academic outcomes.

1.2 Dataset Features

Academic & Study-Related

- student_id
- major
- study_hours_per_day
- attendance_percentage
- previous_gpa
- semester

- study_environment
- access_to_tutoring
- time_management_score
- exam_score

Psychological & Health Factors

- mental_health_rating
- stress_level
- exam_anxiety_score
- motivation_level
- sleep_hours
- diet_quality
- exercise_frequency

Lifestyle & Demographics

- age
- gender
- part_time_job
- extracurricular_participation
- social_activity
- learning_style
- dropout_risk

Family & Support System

- parental_education_level
- family_income_range
- parental_support_level

Technology & Screen Usage

- social_media_hours
- netflix_hours
- screen_time
- internet_quality

CODE:

```
import pandas as pd

df = pd.read_csv('/content/drive/MyDrive/student_performance_dataset.csv')
```

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

df

OUTPUT:

Figure 1: Dataset Features

student_age	gender	major	study_hc	social_m	netflix_h	part_time	attendance	sleep_hc	diet_qual	exercise	parental_internet	mental_h	extracur	previous	semester	stress	le_dropout	social_s	screen_t	stud_en	access	family_in	parental	motivati	exam_an	learning_time	ma_exam_score			
100000	26	Male	Comput	7.64537	3	0.1	Yes	70.3	6.2	Poor	3	Some Cr	High	6	Yes	4	5	5.8	No	0	10.9	Co-Leanr	Yes	High	9	7	8	Reading	3	100
100001	28	Male	Arts	5.7	0.5	0.4	No	88.4	7.2	Good	4	PhD	Low	8.8	No	4	7	5.8	No	5	8.3	Co-Leanr	Yes	Low	7	2	10	Reading	6	99
100002	17	Male	Arts	2.4	4.2	0.7	No	62.1	3.2	Good	4	High Sch	Low	5.7	Yes	3.79	4	8	No	5	8	Library	Yes	High	3	9	6	Kinesthe	7.5	98
100003	27	Other	Psychok	3.4	4.6	2.3	Yes	79.3	4.2	Fair	3	Master	Medium	8.5	Yes	4	6	4.6	No	3	11.7	Co-Leanr	Yes	Low	5	3	10	Reading	3.2	100
100004	25	Female	Business	4.7	0.8	2.7	Yes	62.9	6.5	Good	6	PhD	Low	9.2	No	4	4	5.7	No	2	9.4	Quiet Ro	Yes	Medium	9	1	10	Reading	7.1	98
100005	17	Female	Engineer	4.2	0.5	2.5	No	49.2	3.2	Good	3	PhD	High	9.2	Yes	4	3	15	No	2	8.9	Quiet Ro	No	High	1	10	5	Visual	3.9	94
100006	19	Male	Psychok	4.1	2.4	2.2	Yes	37.9	8.5	Good	6	Some Cr	High	5.8	No	4	2	2.5	No	3	9.2	Dorm	Yes	High	5	7	8	Kinesthe	10	97
100007	17	Male	Psychok	4.1	3.2	0.9	No	44.3	7.4	Fair	5	Master	High	4.9	Yes	3.3	2	5.3	No	3	10	Co-Leanr	Yes	Low	9	3	10	Kinesthe	7.5	84
100008	28	Female	Psychok	0.6	0.3	2.6	Yes	64.2	7.6	Fair	0	High Sch	Medium	3.5	No	2.61	2	4	No	3	4.8	Library	No	Low	4	4	10	Kinesthe	5.8	70
100009	19	Male	Engineer	5.2	1.1	1.6	Yes	40.1	7.2	Fair	6	Master	Low	5.9	Yes	3.78	2	3.3	No	1	8	Cafe	Yes	Medium	7	9	6	Auditory	7	100
100010	16	Other	Arts	0	2.7	0.2	No	44.1	6.9	Poor	2	Some Cr	High	7.8	Yes	4	1	3.7	No	3	3.9	Co-Leanr	Yes	Medium	2	10	5	Visual	3.2	100
100011	26	Other	Biology	3.9	3.2	1.8	No	40.6	8.1	Good	1	PhD	Low	4	Yes	4	6	2	No	0	10.8	Dorm	No	Low	2	9	6	Auditory	2.9	100
100012	19	Female	Business	4.7	3.3	3.9	No	93.4	7.4	Good	1	Some Cr	High	5.6	No	4	3	3.8	No	2	12.1	Dorm	Yes	Medium	5	10	5	Auditory	2.9	96
100013	24	Female	Business	7.4	3.2	1	Yes	73.7	5.9	Poor	6	Some Ct	Low	8.1	No	3.31	1	5.9	No	2	13.1	Library	No	High	1	6	9	Kinesthe	6.2	83
100014	21	Male	Business	5.5	3.1	0.6	Yes	98.6	8	Good	2	Master	High	7.3	Yes	3.21	4	7.5	No	5	11.1	Cafe	No	Low	3	6	9	Auditory	8	77
100015	17	Female	Comput	3.5276	0.1	0.8	Yes	98	8	Fair	1	PhD	Medium	10	Yes	3.14	8	4.7	No	2	5.1	Quiet Ro	No	Medium	9	6	9	Reading	9.5	87
100016	16	Male	Biology	5	4.5	0.9	No	40.1	9.7	Good	6	PhD	High	9.9	No	4	7	5.9	No	0	10.4	Dorm	No	High	4	6	9	Reading	4.9	100
100017	27	Female	Biology	3.7	2.1	3.9	No	76.9	8.9	Fair	2	PhD	High	5.8	No	3.6	5	4.9	No	3	11	Co-Leanr	No	Low	5	5	10	Reading	2.9	93
100018	26	Male	Biology	2.6	0.5	3.8	Yes	53.5	9.1	Fair	4	High Sch	High	9.6	Yes	3.77	8	6.2	No	5	7.4	Quiet Ro	No	Low	4	8	7	Kinesthe	6.5	88
100019	25	Male	Arts	3.9	3.8	3.1	No	53.1	5.4	Good	6	PhD	Low	7.9	No	3.7	7	2.4	No	3	11	Quiet Ro	Yes	Medium	1	9	6	Kinesthe	3.2	92
100020	26	Other	Biology	3.9	1.3	1	No	77.6	6.4	Fair	7	High Sch	Low	8.1	No	2.65	6	5	No	1	7.5	Dorm	No	High	5	4	10	Visual	3.4	60
100021	22	Other	Psychok	2	1	1.7	Yes	91.4	5.2	Good	0	Master	Medium	6.4	No	3.33	4	4.9	No	5	6.5	Cafe	No	Medium	1	6	9	Kinesthe	3.7	92
100022	24	Other	Psychok	5.8	3.4	1.6	Yes	95.2	7.8	Good	7	PhD	Medium	7.3	No	4	8	6.8	No	1	11.1	Quiet Ro	No	Low	7	10	5	Visual	6.1	100
100023	23	Male	Comput	3.27913	3.8	3.3	No	68.2	6.5	Good	1	High Sch	Medium	9.3	No	4	5	6.1	No	1	10.9	Quiet Ro	Yes	Medium	4	2	10	Visual	8	100
100024	16	Other	Psychok	6.1	1.3	1.5	Yes	86.7	5.2	Poor	1	Bachelor	High	1.8	No	3.27	4	9.5	Yes	5	10.1	Co-Leanr	Yes	Low	10	1	10	Kinesthe	9.4	80
100025	19	Male	Biology	4.4	2.1	1.5	Yes	40.9	7.7	Fair	5	Master	High	7.4	No	3.76	4	6.6	No	5	9	Cafe	Yes	Low	7	6	9	Auditory	6.7	89
100026	24	Other	Business	6.2	0.4	3.9	No	74.2	4	Good	7	Bachelor	Medium	10	No	4	9	3.8	No	2	10.6	Library	No	Low	9	6	9	Reading	2.6	92
100027	20	Other	Arts	3.6	3.8	2.8	Yes	66.9	7.8	Poor	7	Bachelor	Low	8.7	Yes	3.75	5	7.1	No	4	11.5	Co-Leanr	Yes	Low	7	4	10	Reading	3.8	88
100028	21	Female	Arts	4	1	0.5	No	51.5	7.7	Good	2	Bachelor	Medium	4.7	No	3.68	6	5.3	No	2	6.6	Quiet Ro	No	High	10	9	6	Auditory	6.4	95
100029	19	Female	Comput	3.83787	3.8	0.4	No	68.2	5.8	Fair	4	Some Cr	High	4.5	Yes	4	3	3.9	No	4	8.1	Quiet Ro	Yes	Low	5	8	7	Kinesthe	2	100
100030	26	Other	Comput	7.60632	4	0.9	Yes	94.5	6.9	Fair	7	PhD	Low	5.1	No	4	3	7.8	No	1	13.6	Co-Leanr	Yes	Medium	5	10	5	Visual	6.6	100
100031	26	Male	Business	4.1	2.3	2.4	Yes	57.3	6.8	Fair	3	Master	High	5.2	Yes	3.73	2	4.3	No	4	9.4	Co-Leanr	Yes	Low	4	5	10	Auditory	8.1	93
100032	19	Other	Psychok	4.7	4.1	3.7	Yes	81.3	7.1	Good	7	Bachelor	Low	6.6	Yes	4	5	4.3	No	5	13.4	Co-Leanr	Yes	High	9	8	7	Visual	4.9	94
100033	21	Other	Business	5.1	3.8	0.7	Yes	95	5.3	Fair	7	PhD	High	5.4	Yes	3.28	1	6.9	No	0	10.6	Cafe	Yes	Medium	8	2	10	Reading	5.2	76
100034	18	Female	Engineer	5.3	0.6	3.1	No	79.5	7.5	Good	0	Master	Medium	6.7	No	4	8	3.9	No	1	10.7	Cafe	Yes	Low	4	9	6	Kinesthe	4.3	100
100035	20	Other	Psychok	6.7	1.2	0.6	No	45.6	6.3	Good	6	High Sch	Low	7.9	No	3.59	5	5.1	No	3	9.6	Dorm	Yes	Medium	2	3	10	Reading	5.5	99
100036	23	Male	Engineer	3.4	2.1	3.7	No	57	6.4	Fair	0	PhD	Low	8.8	Yes	3.08	3	5.9	No	4	10.2	Cafe	Yes	High	8	3	10	Reading	6.7	91
100037	22	Female	Business	7.2	3.5	3.2	Yes	69.7	7.9	Poor	2	PhD	Medium	4.5	No	4	4	3.3	No	3	14.7	Cafe	Yes	High	6	2	10	Visual	8.9	100
100038	16	Other	Psychok	2.4	3.2	3.4	Yes	97.2	9.4	Good	6	Master	Medium	9.8	Yes	4	7	2.4	No	3	10.7	Dorm	No	High	1	10	5	Auditory	1.6	100
100039	17	Other	Arts	7.4	1.5	1.5	No	74	7.4	Good	6	Some Ct	Low	3.8	Yes	4	2	7.9	No	4	10.7	Co-Leanr	No	Low	2	10	5	Auditory	8.8	100
100040	16	Male	Engineer	6.4	0.8	1.9	No	78.3	8.9	Fair	4	Bachelor	Medium	6	No	3.36	5	8.4	No	4	9.7	Cafe	Yes	Medium	5	4	10	Visual	7	90
100041	21	Other	Business	1.2	3.7	0.2	Yes	90	6.8	Good	7	Some Ct	Low	8.1	No	3.75	5	5.6	No	5	6.3	Co-Leanr	No	High	6	4	10	Reading	6.7	91
100042	28	Other	Arts	5.9	0.7	3.6	Yes	50.5	6.9	Poor	6	PhD	Medium	7.3	Yes	4	3	6	No	3	10.9	Library	No	Medium	9	9	6	Reading	9.2	100

80000 rows × 31 columns

CHAPTER 3:

DATA CLEANING

3.1. Dropped Irrelevant Column

The " student_id " column was removed as it serves only as a unique identifier and does not contribute to model training or predictions.

CODE:

```
{"type":"dataframe","variable_name":"df"}  
  
df.drop('student_id',inplace=True,axis=1)
```

3.2. Checked for Missing Values

Used the **missingno** library to visualize missing data.

Generated a **bar plot** to confirm data completeness.

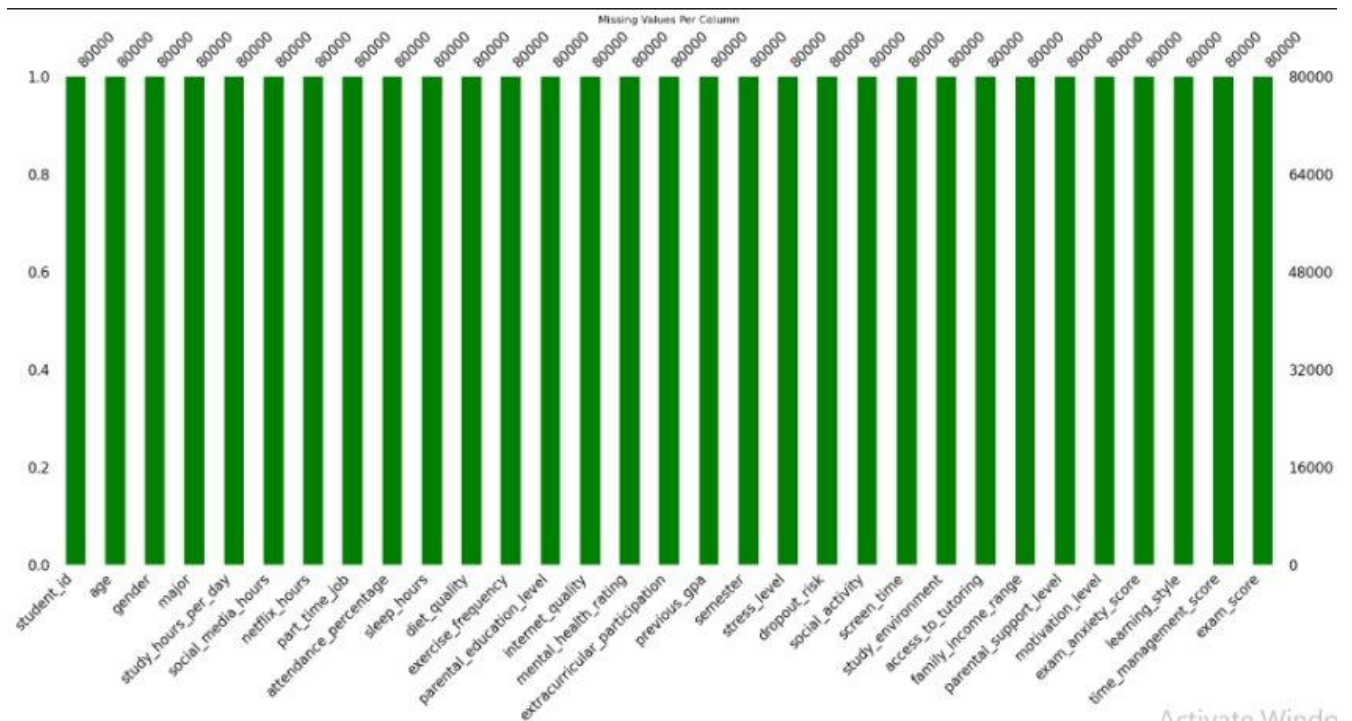
Result: No missing values found — the dataset is **synthetically generated** and complete.

Missing value

CODE:

```
!pip install missingno  
  
import missingno as mn  
import matplotlib.pyplot as plt  
  
# Bar plot of missing values per column  
mn.bar(df)  
plt.title("Missing Values Per Column")  
plt.show()
```

Figure 2: Bar plot of missing values per column



Interpretation: This bar plot, generated using the missingno library, visually confirms that the dataset contains no missing values across all columns. Each bar represents a column with a height corresponding to the number of non-null entries (80,000), indicating that the synthetically generated dataset is complete and ready for analysis without requiring imputation or additional data cleaning for missing values.

```
df.isnull().sum()
```

```
age 0
gender 0
major 0
study_hours_per_day 0
social_media_hours 0
netflix_hours 0
part_time_job 0
attendance_percentage 0
sleep_hours 0
diet_quality 0
exercise_frequency 0
parental_education_level 0
internet_quality 0
mental_health_rating 0
extracurricular_participation 0
```

```

previous_gpa          0
semester              0
stress_level          0
dropout_risk          0
social_activity        0
screen_time           0
study_environment     0
access_to_tutoring    0
family_income_range   0
parental_support_level 0
motivation_level      0
exam_anxiety_score    0
learning_style        0
time_management_score  0
exam_score            0
dtype: int64

```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 80000 entries, 0 to 79999
```

```
Data columns (total 30 columns):
```

#	Column	Non-Null Count	Dtype
0	age	80000 non-null	int64
1	gender	80000 non-null	object
2	major	80000 non-null	object
3	study_hours_per_day	80000 non-null	float64
4	social_media_hours	80000 non-null	float64
5	netflix_hours	80000 non-null	float64
6	part_time_job	80000 non-null	object
7	attendance_percentage	80000 non-null	float64
8	sleep_hours	80000 non-null	float64
9	diet_quality	80000 non-null	object
10	exercise_frequency	80000 non-null	int64
11	parental_education_level	80000 non-null	object
12	internet_quality	80000 non-null	object
13	mental_health_rating	80000 non-null	float64
14	extracurricular_participation	80000 non-null	object
15	previous_gpa	80000 non-null	float64
16	semester	80000 non-null	int64
17	stress_level	80000 non-null	float64
18	dropout_risk	80000 non-null	object
19	social_activity	80000 non-null	int64
20	screen_time	80000 non-null	float64
21	study_environment	80000 non-null	object
22	access_to_tutoring	80000 non-null	object
23	family_income_range	80000 non-null	object
24	parental_support_level	80000 non-null	int64

```
25 motivation_level      80000 non-null int64
26 exam_anxiety_score     80000 non-null int64
27 learning_style         80000 non-null object
28 time_management_score   80000 non-null float64
29 exam_score             80000 non-null int64
dtypes: float64(10), int64(8), object(12)
memory usage: 18.3+ MB
```


CHAPTER 4:

EXPLORATORY DATA ANALYSIS (EDA)

4.1 Summary Statistics of the Dataset

```
df.describe() # this is only for numerical columns
```

Figure 3: Summary Statistics of the Dataset

Feature	Mean	Std Dev	Min	25%	50% (Median)	75%	Max
Age	22.00	3.75	16.00	19.00	22.00	25.00	28.00
Study Hours/Day	4.17	2.00	0.00	2.80	4.13	5.50	12.00
Social Media Hours	2.50	1.45	0.00	1.20	2.50	3.80	5.00
Netflix Hours	2.00	1.16	0.00	1.00	2.00	3.00	4.00
Attendance %	69.97	17.33	40.00	55.00	69.90	84.90	100.0
Sleep Hours	7.02	1.47	4.00	6.00	7.00	8.00	12.00
Exercise Frequency	3.52	2.29	0.00	2.00	4.00	6.00	7.00
Mental Health Rating	6.80	1.92	1.00	5.50	6.90	8.20	10.00
Previous GPA	3.60	0.46	1.64	3.27	3.79	4.00	4.00
Semester	4.50	2.30	1.00	2.00	5.00	7.00	8.00
Stress Level	5.01	1.95	1.00	3.60	5.00	6.40	10.00
Social Activity	2.50	1.70	0.00	1.00	2.00	4.00	5.00
Screen Time (hrs/day)	9.67	2.78	0.30	7.80	9.70	11.60	21.00
Parental Support Level	5.48	2.87	1.00	3.00	5.00	8.00	10.00
Motivation Level	5.49	2.87	1.00	3.00	5.00	8.00	10.00
Exam Anxiety Score	8.51	1.80	5.00	7.00	10.00	10.00	10.00
Time Management Score	5.50	2.60	1.00	3.20	5.50	7.80	10.00
Exam Score	89.14	11.59	36.00	82.00	93.00	100.0	100.0

4.2 Univariate Analysis

```
df.select_dtypes(include = 'number').columns
```

```
Index(['age', 'study_hours_per_day', 'social_media_hours', 'netflix_hours',  
      'attendance_percentage', 'sleep_hours', 'exercise_frequency',  
      'mental_health_rating', 'previous_gpa', 'semester', 'stress_level',  
      'social_activity', 'screen_time', 'parental_support_level',  
      'motivation_level', 'exam_anxiety_score', 'time_management_score',  
      'exam_score'],  
      dtype='object')
```

```
len(df.select_dtypes(include = 'number').columns) # total numerical columns
```

18

```
df.select_dtypes(include = 'object').columns
```

```
Index(['gender', 'major', 'part_time_job', 'diet_quality',  
      'parental_education_level', 'internet_quality',  
      'extracurricular_participation', 'dropout_risk', 'study_environment',  
      'access_to_tutoring', 'family_income_range', 'learning_style'],  
      dtype='object')
```

```
len(df.select_dtypes(include = 'object').columns) # total categorical column
```

12

```

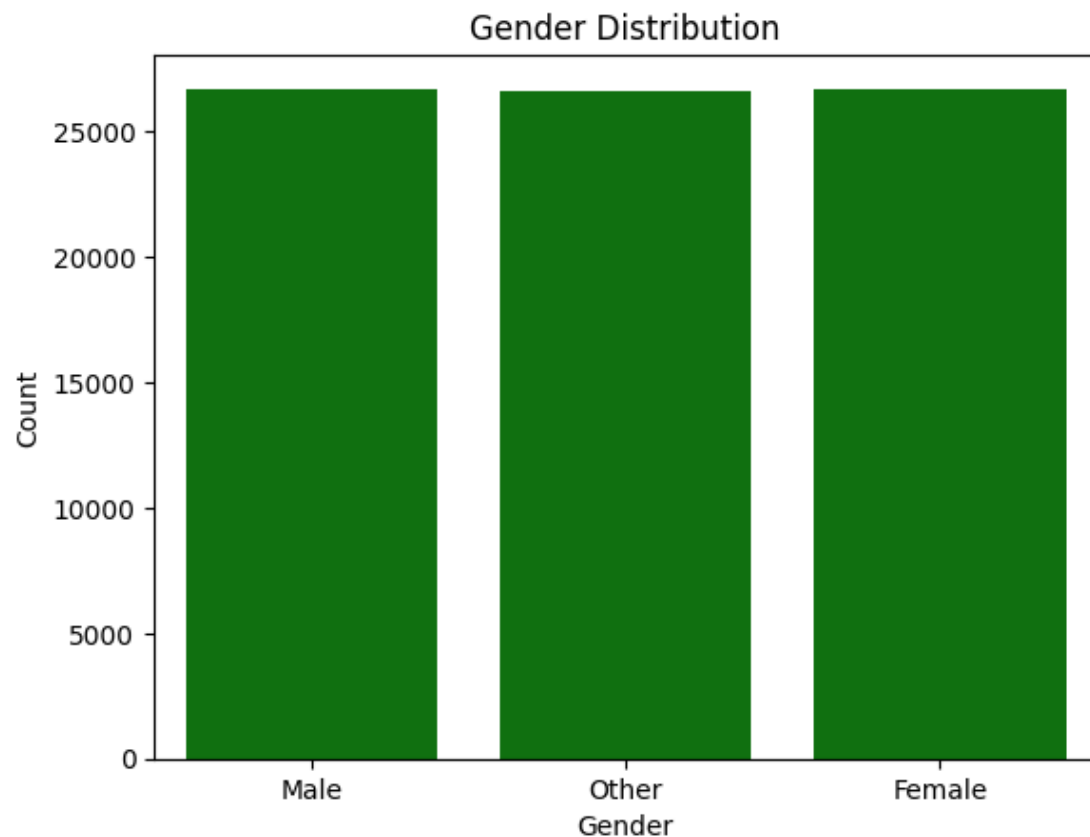
gender
import seaborn as sns

import matplotlib.pyplot as plt

sns.countplot(data=df, x='gender', color = 'g')
plt.title('Gender Distribution')
plt.xlabel('Gender')
plt.ylabel('Count')
plt.show()

```

Figure 4: Gender Distribution



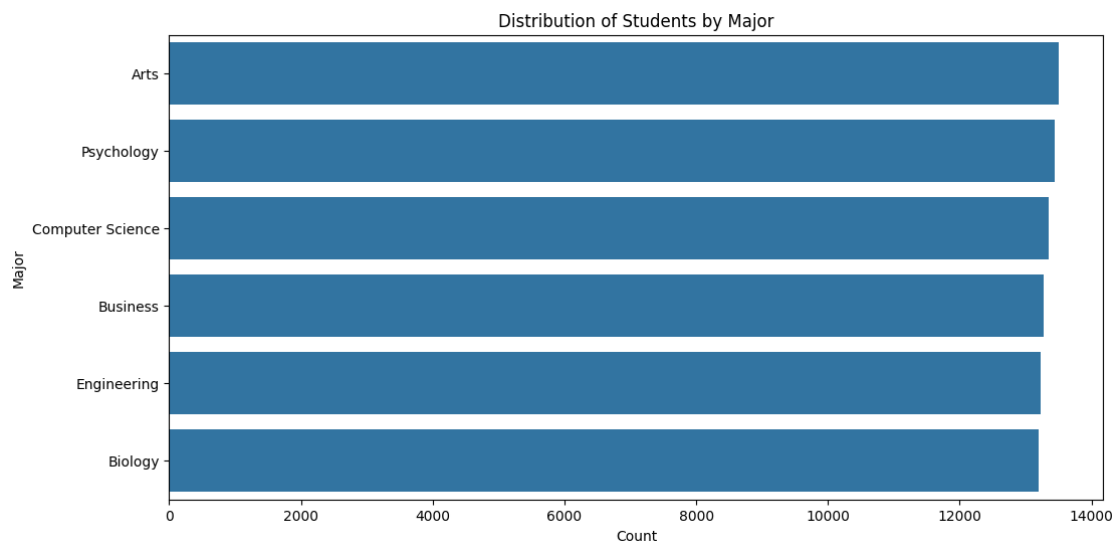
Interpretation: The count plot shows a balanced distribution of gender across the dataset, with male, female, and other categories represented nearly equally. This suggests that the dataset captures a diverse student population, allowing for unbiased analysis of gender's impact on academic performance.

major

```
plt.figure(figsize=(12, 6))
sns.countplot(data=df, y='major', order=df['major'].value_counts().index)
plt.title('Distribution of Students by Major')
plt.xlabel('Count')
plt.ylabel('Major')
plt.show()

print(df['major'].value_counts())
```

Figure 5: Distribution of Students by Major



```
major
Arts          13505
Psychology    13437
Computer Science 13352
Business      13276
Engineering   13229
Biology       13201
Name: count, dtype: int64
```

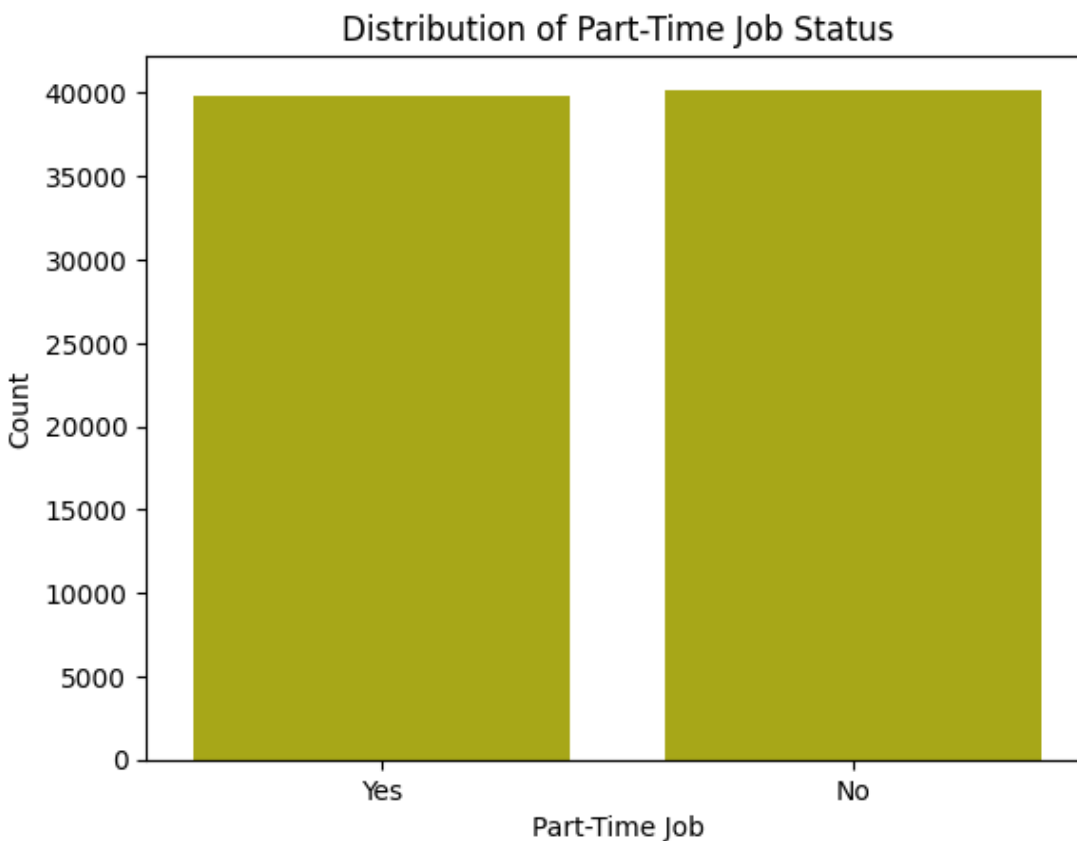
Interpretation: This count plot illustrates the distribution of students across six majors (Arts, Psychology, Computer Science, Business, Engineering, Biology), with each major having a similar count (~13,000–13,500). The balanced representation ensures that no single major dominates the dataset, enabling fair comparisons across academic disciplines.

part_time_job

```
print(df['part_time_job'].value_counts())  
sns.countplot(data=df, x='part_time_job', color = 'y')  
plt.title('Distribution of Part-Time Job Status')  
plt.xlabel('Part-Time Job')  
plt.ylabel('Count')  
plt.show()
```

```
part_time_job  
No      40195  
Yes     39805  
Name: count, dtype: int64
```

Figure 6: Distribution of Part-Time Job Status



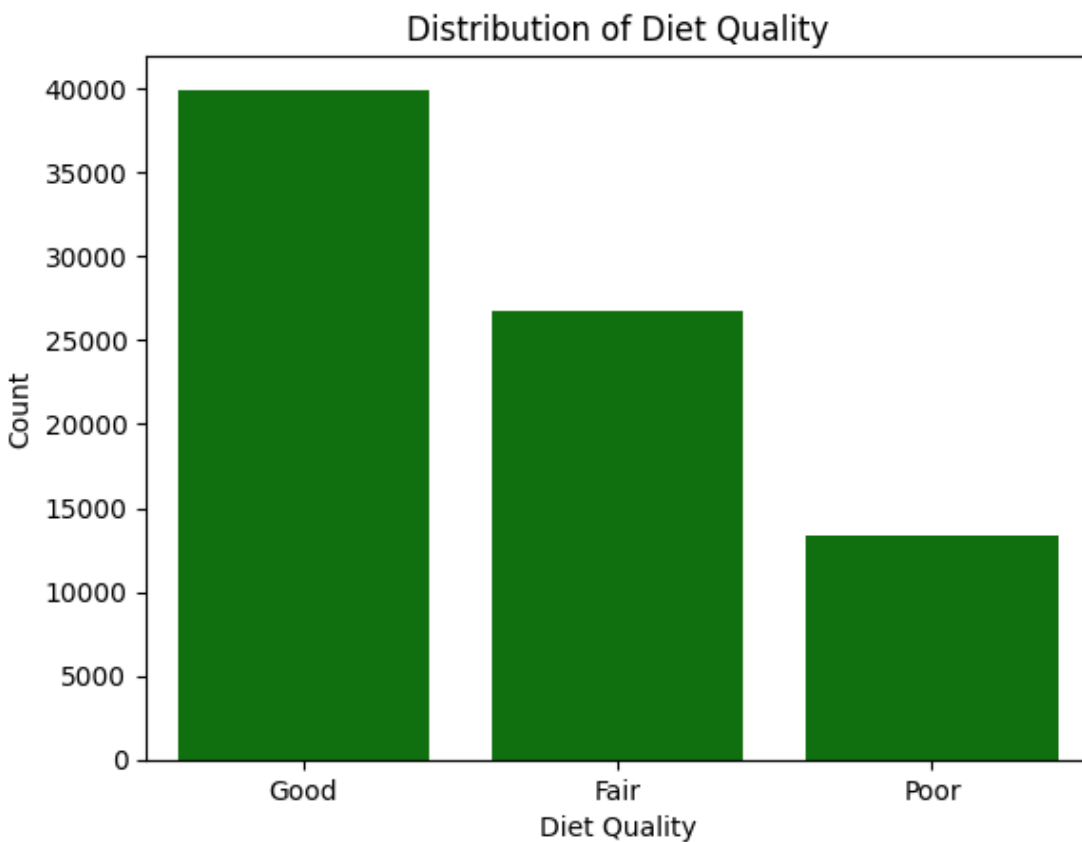
Interpretation: The count plot reveals an almost equal split between students with part-time jobs (39,805) and those without (40,195). This near-even distribution allows for robust analysis of how part-time work might influence academic outcomes without skewing results due to imbalanced representation.

diet_quality

```
print(df['diet_quality'].value_counts())  
sns.countplot(data=df, x='diet_quality',  
order=df['diet_quality'].value_counts().index, color = 'g')  
plt.title('Distribution of Diet Quality')  
plt.xlabel('Diet Quality')  
plt.ylabel('Count')  
plt.show()
```

```
diet_quality  
Good      39935  
Fair      26713  
Poor      13352  
Name: count, dtype: int64
```

Figure 7: Distribution of Diet Quality



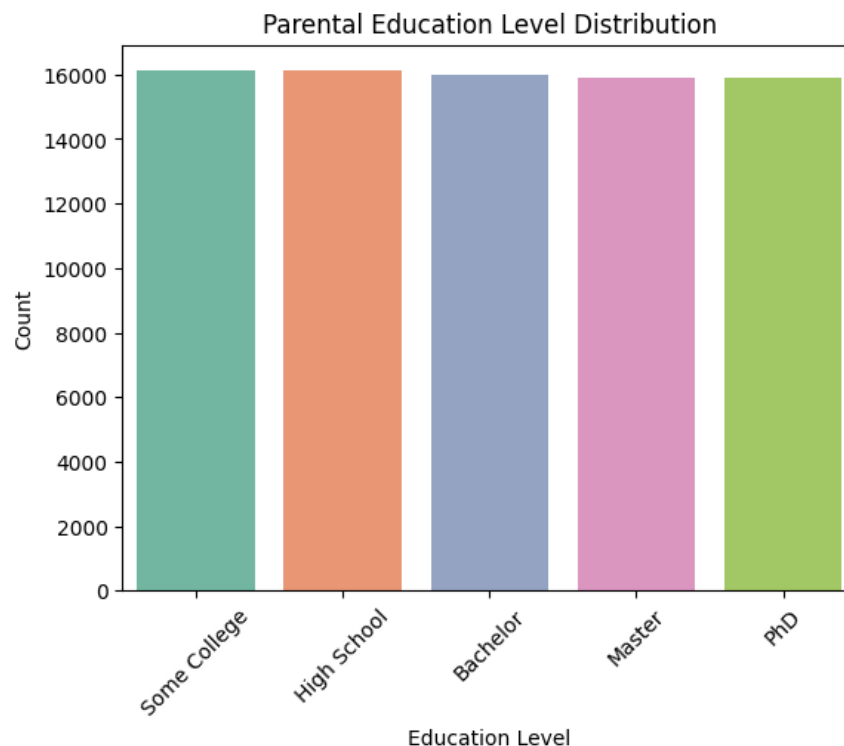
Interpretation: The count plot shows diet quality distribution, with "Good" (39,935) being the most common, followed by "Fair" (26,713) and "Poor" (13,352). The higher prevalence of good diet quality suggests a generally healthy student population, but the presence of poorer diets allows exploration of its impact on performance.

parental_education_level

```
print(df['parental_education_level'].value_counts())
sns.countplot(data=df, x='parental_education_level',
order=df['parental_education_level'].value_counts().index, color =
'b',palette='Set2')
plt.title("Parental Education Level Distribution")
plt.xlabel("Education Level")
plt.ylabel("Count")
plt.xticks(rotation=45)
plt.show()
```

```
parental_education_level
Some College    16115
High School    16106
Bachelor       15986
Master         15908
PhD            15885
Name: count, dtype: int64
```

Figure 8: Distribution of Parental Education Level



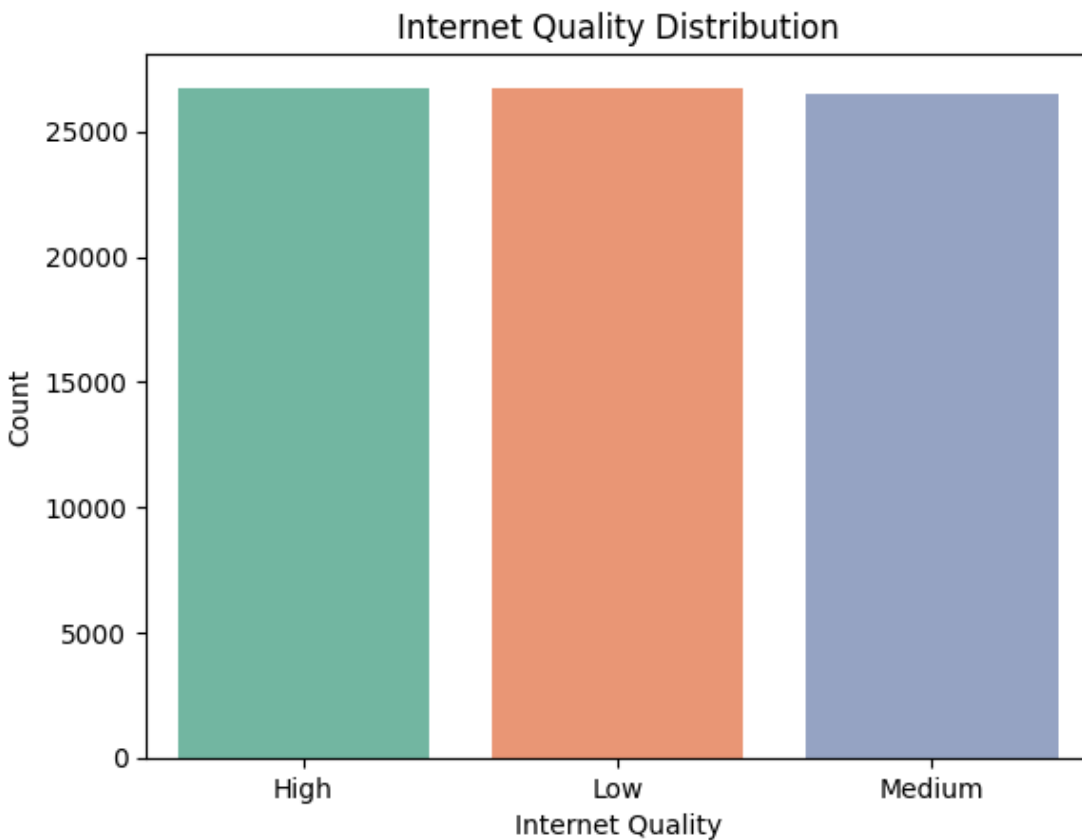
Interpretation: This count plot displays a balanced distribution of parental education levels (High School, Some College, Bachelor, Master, PhD), each with ~15,800–16,100 occurrences. This even spread ensures that the dataset captures diverse socio-economic backgrounds, facilitating analysis of parental education's influence on student outcomes.

internet_quality

```
print(df['internet_quality'].value_counts())
sns.countplot(data=df, x='internet_quality',
order=df['internet_quality'].value_counts().index,
palette='Set2', legend=False)
plt.title('Internet Quality Distribution')
plt.xlabel('Internet Quality')
plt.ylabel('Count')
plt.show()
```

```
internet_quality
High      26749
Low       26714
Medium    26537
Name: count, dtype: int64
```

Figure 9: Distribution of Internet Quality



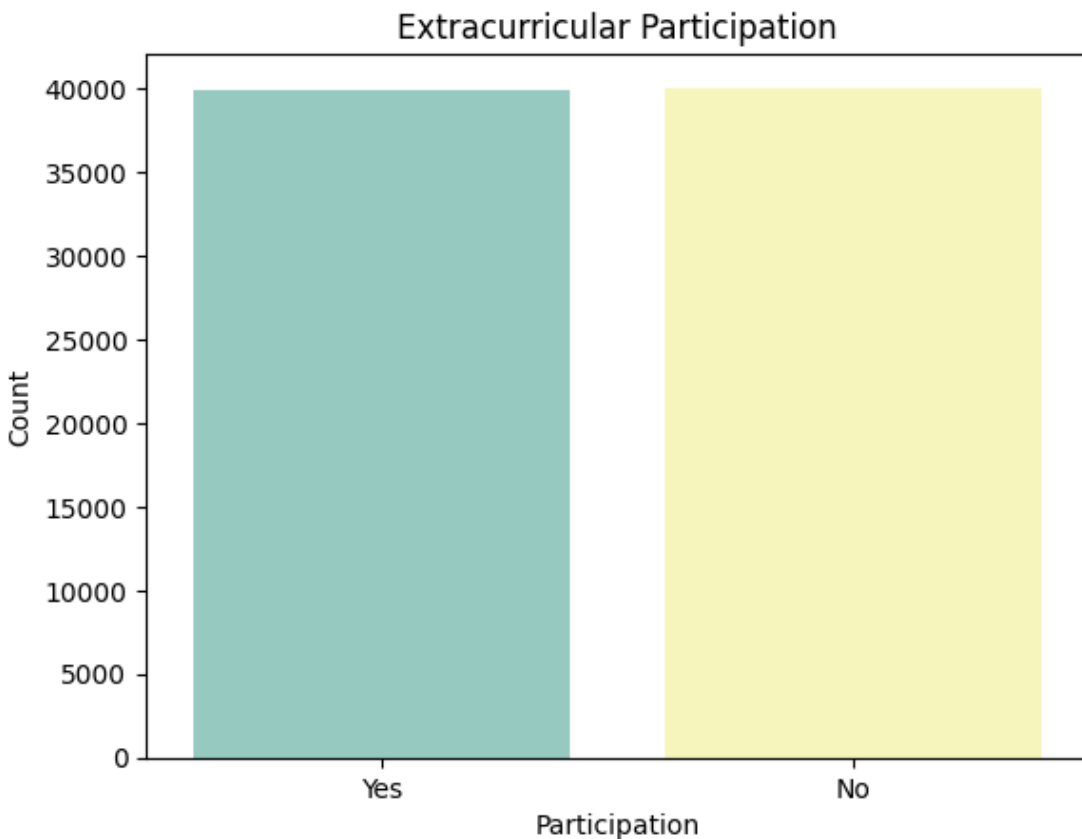
Interpretation: The count plot indicates that internet quality is evenly distributed across "High" (26,749), "Medium" (26,537), and "Low" (26,714) categories. This balance allows for meaningful analysis of how internet access quality affects academic performance, especially in technology-dependent learning environments.

extracurricular_participation

```
print(df['extracurricular_participation'].value_counts())
sns.countplot(data=df, x='extracurricular_participation', palette='Set3')
plt.title('Extracurricular Participation')
plt.xlabel('Participation')
plt.ylabel('Count')
plt.show()
```

```
extracurricular_participation
No      40058
Yes     39942
Name: count, dtype: int64
```

Figure 10: Distribution of Extracurricular Participation



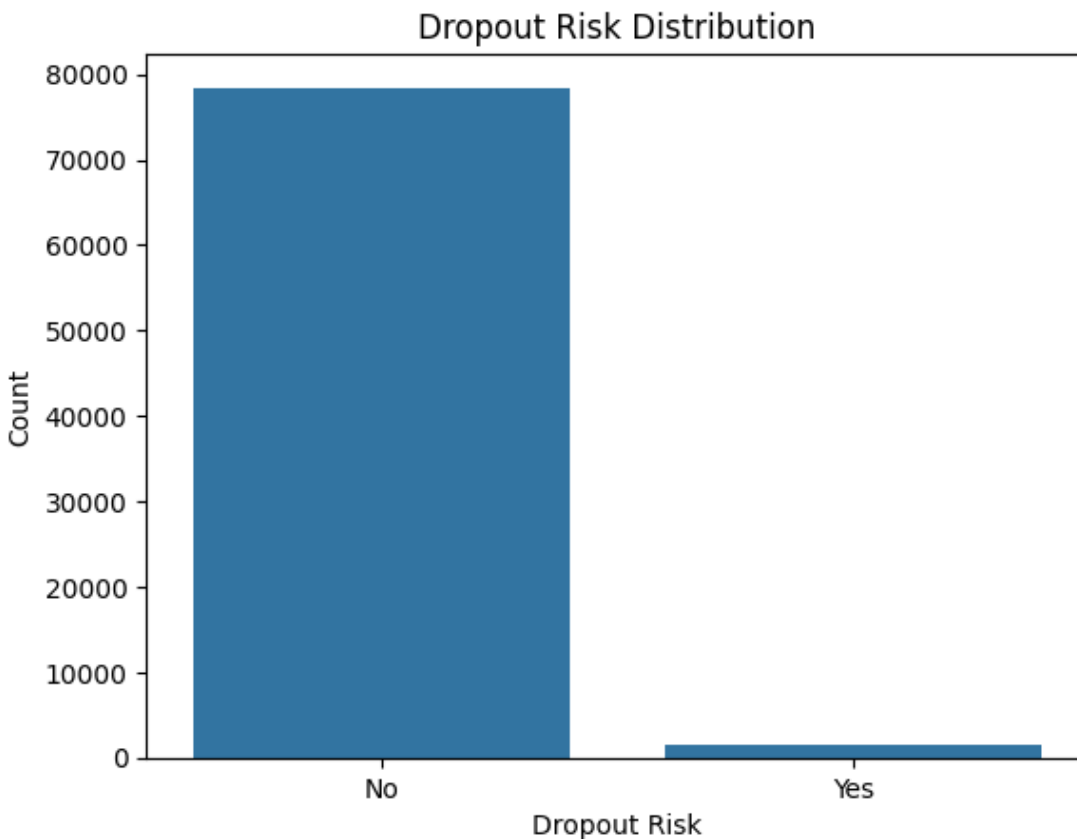
Interpretation: The count plot shows a near-equal split between students participating in extracurricular activities (39,942) and those who do not (40,058). This balanced distribution supports analysis of how extracurricular involvement impacts academic performance without bias from uneven representation.

dropout_risk

```
print(df['dropout_risk'].value_counts())
sns.countplot(data=df, x='dropout_risk')
plt.title('Dropout Risk Distribution')
plt.xlabel('Dropout Risk')
plt.ylabel('Count')
plt.show()
```

```
dropout_risk
No      78418
Yes     1582
Name: count, dtype: int64
```

Figure 11: Distribution of Dropout Risk



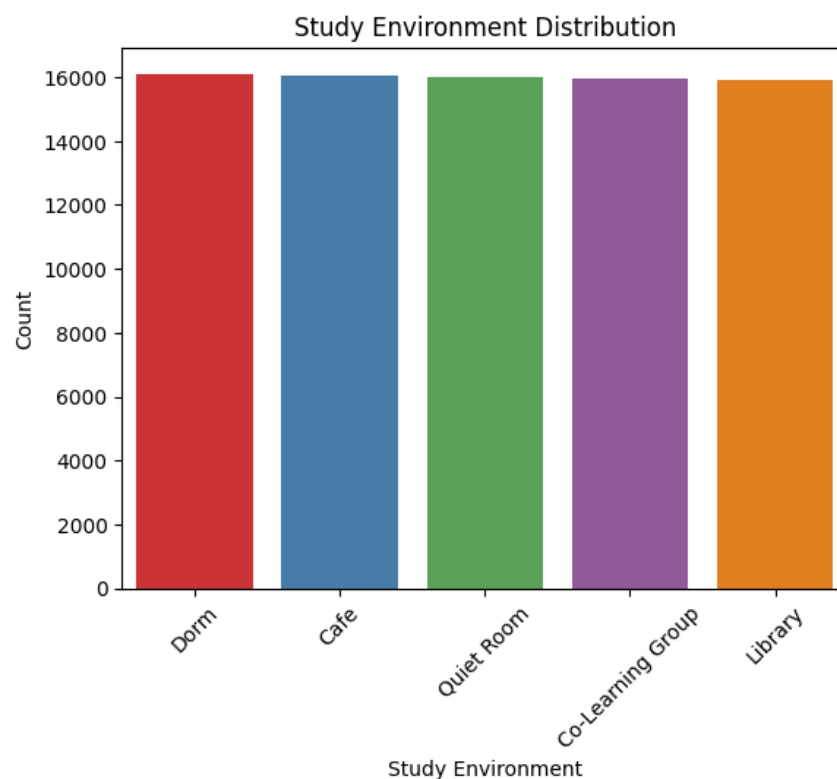
Interpretation: The count plot highlights a significant imbalance, with 78,418 students at no risk of dropping out and only 1,582 at risk. This skewed distribution suggests that dropout risk is relatively low in the dataset but allows for targeted analysis of factors contributing to this critical outcome.

study_environment

```
print(df['study_environment'].value_counts())
sns.countplot(data=df, x='study_environment',
order=df['study_environment'].value_counts().index, palette='Set1')
plt.title('Study Environment Distribution')
plt.xlabel('Study Environment')
plt.ylabel('Count')
plt.xticks(rotation=45)
plt.show()
```

```
study_environment
Dorm                16119
Cafe                16038
Quiet Room          15991
Co-Learning Group   15949
Library             15903
Name: count, dtype: int64
```

Figure 12: Distribution of Study Environment



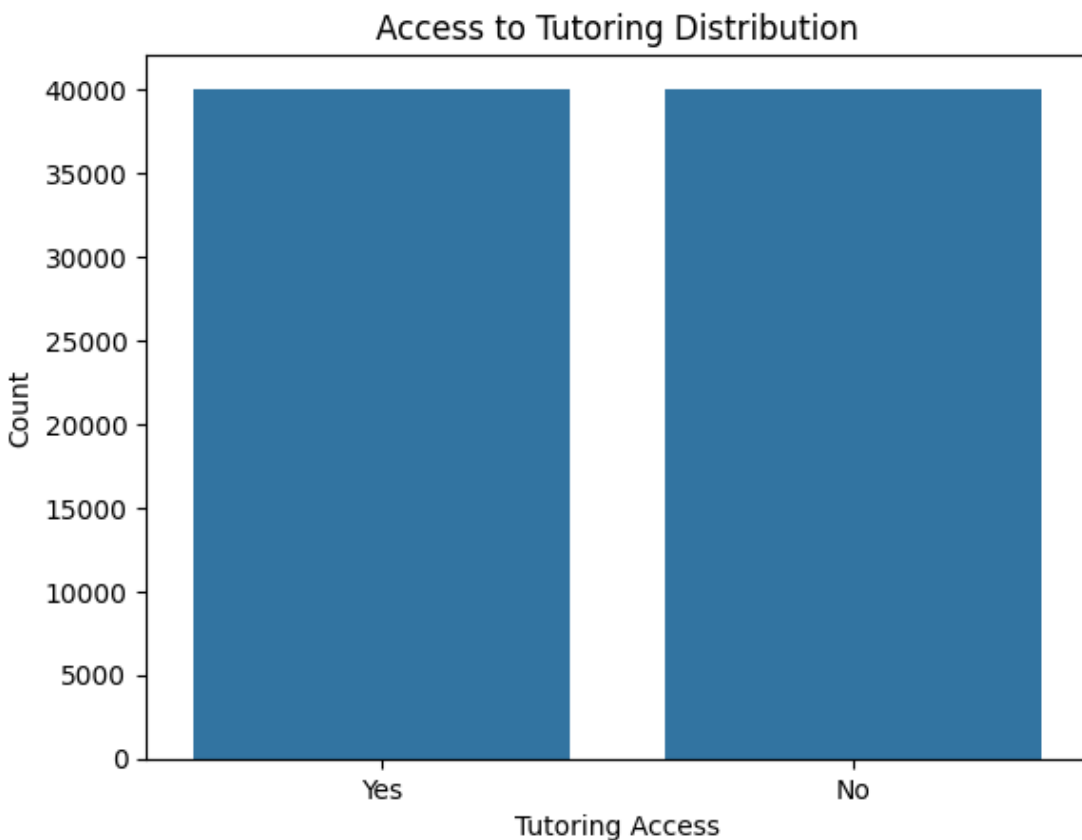
Interpretation: The count plot shows a balanced distribution of study environments (Dorm, Cafe, Quiet Room, Co-Learning Group, Library), each with ~15,900–16,100 students. This variety enables exploration of how different study settings influence academic performance, with no single environment dominating.

access_to_tutoring

```
print(df['access_to_tutoring'].value_counts())  
sns.countplot(data=df, x='access_to_tutoring')  
plt.title('Access to Tutoring Distribution')  
plt.xlabel('Tutoring Access')  
plt.ylabel('Count')  
plt.show()
```

```
access_to_tutoring  
No      40039  
Yes     39961  
Name: count, dtype: int64
```

Figure 13: Distribution of Access to Tutoring



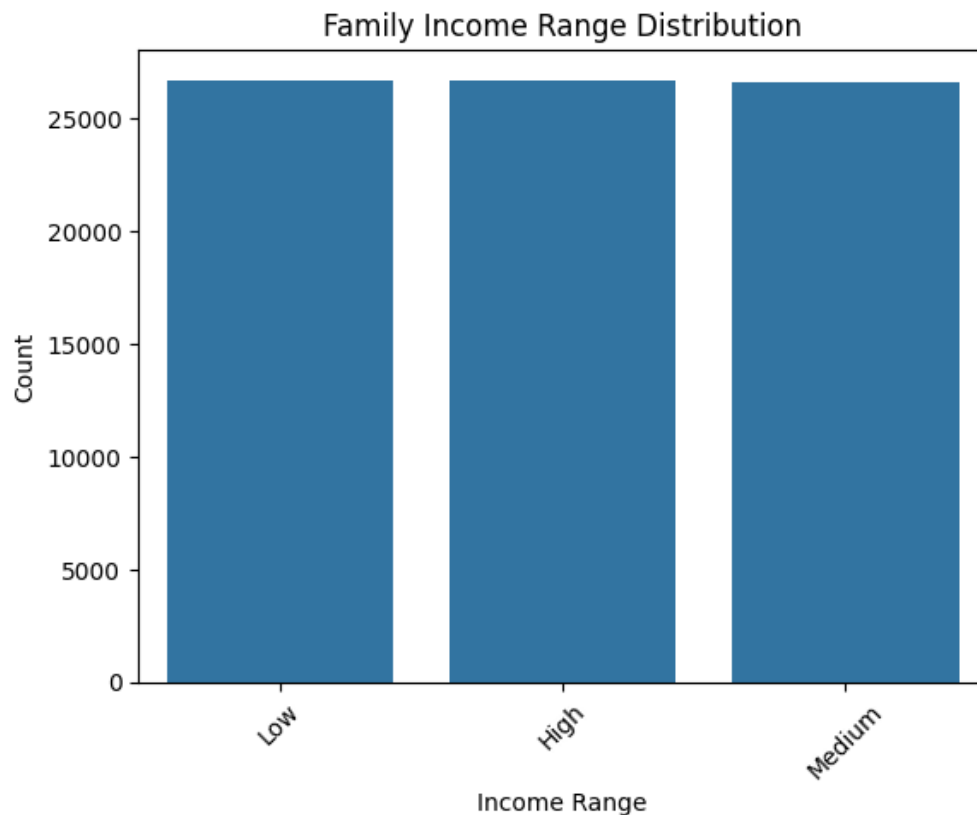
Interpretation: The count plot indicates a near-even split between students with access to tutoring (39,961) and those without (40,039). This balance allows for a fair assessment of tutoring's impact on academic success, reflecting diverse access to academic support

family_income_range

```
print(df['family_income_range'].value_counts())
sns.countplot(data=df, x='family_income_range',
order=df['family_income_range'].value_counts().index)
plt.title('Family Income Range Distribution')
plt.xlabel('Income Range')
plt.ylabel('Count')
plt.xticks(rotation=45)
plt.show()
```

```
family_income_range
Low      26720
High     26653
Medium   26627
Name: count, dtype: int64
```

Figure 14: Distribution of Family Income Range



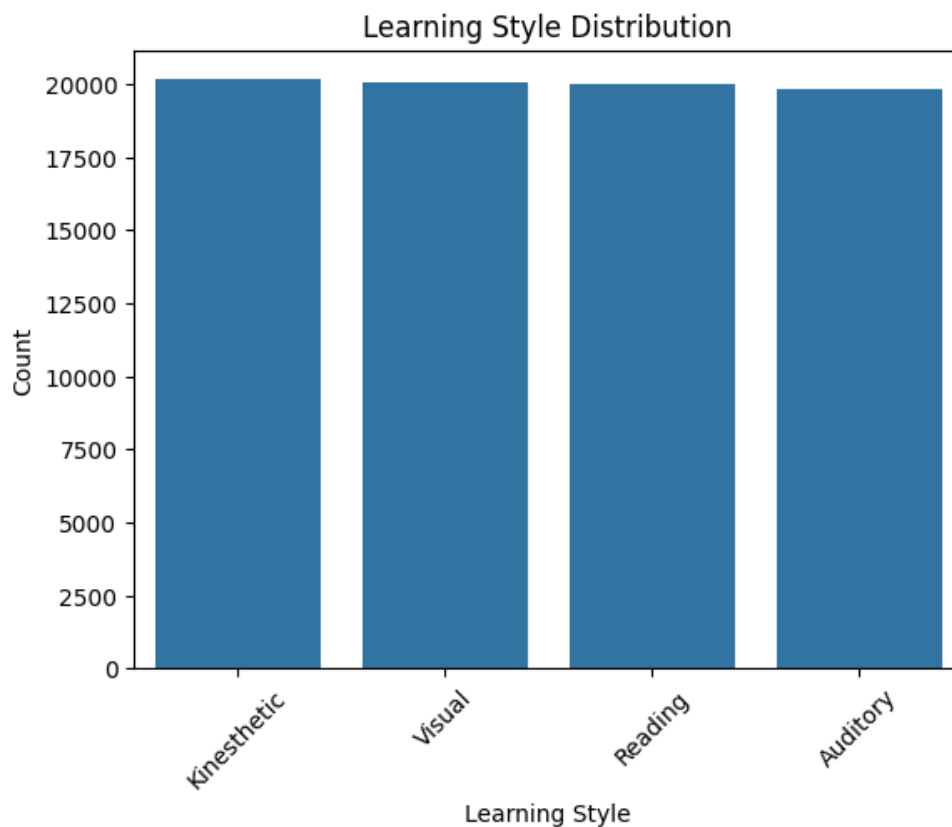
Interpretation: The count plot shows an even distribution across family income ranges (Low: 26,720, Medium: 26,627, High: 26,653). This balanced representation supports analysis of socioeconomic status's effect on academic performance without bias from overrepresented income groups.

learning_style

```
print(df['learning_style'].value_counts())
sns.countplot(data=df, x='learning_style',
order=df['learning_style'].value_counts().index)
plt.title('Learning Style Distribution')
plt.xlabel('Learning Style')
plt.ylabel('Count')
plt.xticks(rotation=45)
plt.show()
```

```
learning_style
Kinesthetic    20143
Visual         20058
Reading        19997
Auditory       19802
Name: count, dtype: int64
```

Figure 15: Distribution of Life Style



Interpretation: The count plot displays a balanced distribution of learning styles (Kinesthetic: 20,143, Visual: 20,058, Reading: 19,997, Auditory: 19,802). This diversity allows for investigation into how different learning preferences influence academic outcomes, with no single style dominating.

Numerical Columns

```
import matplotlib.pyplot as plt
import seaborn as sns

numerical_columns = [
    'age', 'study_hours_per_day', 'social_media_hours', 'netflix_hours',
    'attendance_percentage', 'sleep_hours', 'exercise_frequency',
    'mental_health_rating', 'previous_gpa', 'semester', 'stress_level',
    'social_activity', 'screen_time', 'parental_support_level',
    'motivation_level', 'exam_anxiety_score', 'time_management_score',
    'exam_score'
]

for col in numerical_columns:
    print(f"=== {col.upper()} ===")
    print(df[col].describe())
    print("\n")

    fig, axs = plt.subplots(1, 2, figsize=(12, 4))

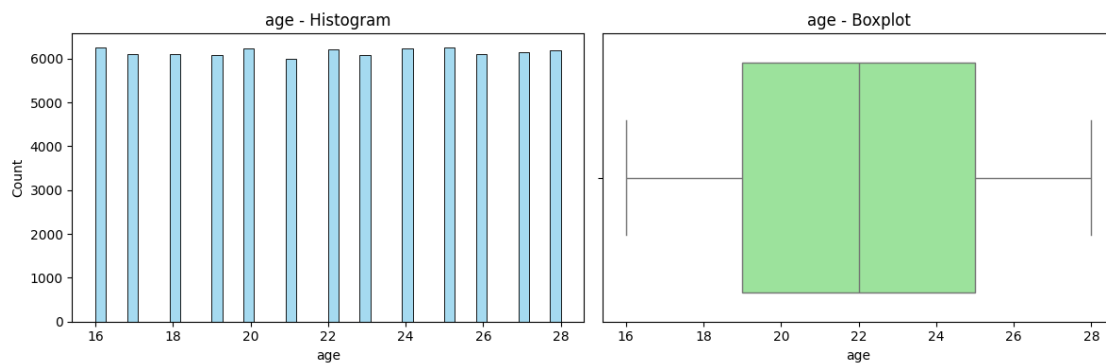
    # Removed KDE curve
    sns.histplot(df[col], kde=False, ax=axs[0], color='skyblue',
edgecolor='black')
    axs[0].set_title(f'{col} - Histogram')

    sns.boxplot(x=df[col], ax=axs[1], color='lightgreen')
    axs[1].set_title(f'{col} - Boxplot')

plt.tight_layout()
plt.show()
```

```
=== AGE ===  
count      80000.000000  
mean       22.004288  
std        3.745570  
min        16.000000  
25%        19.000000  
50%        22.000000  
75%        25.000000  
max        28.000000  
Name: age, dtype: float64
```

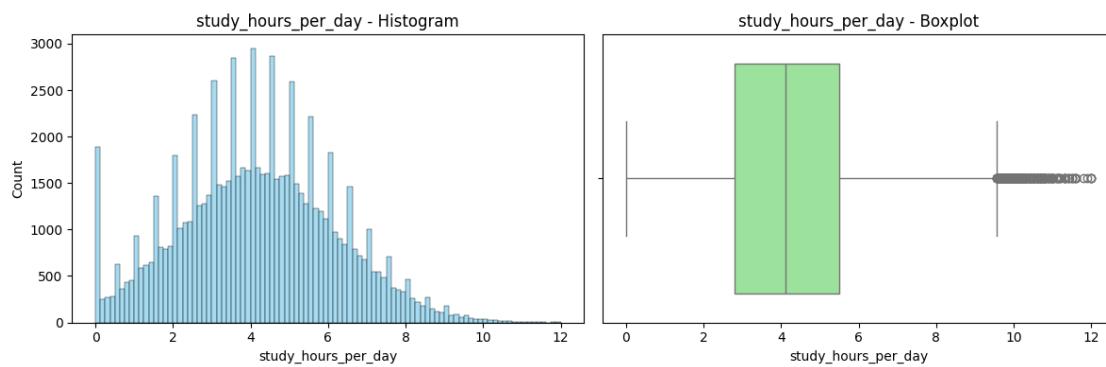
Figure 16: Histogram and Boxplot of 'Age'



Interpretation: The histogram shows a roughly normal distribution of student ages (mean ~22, range 16–28), with a slight right skew. The boxplot confirms no extreme outliers, indicating a typical college-age population, suitable for analyzing age-related effects on performance.


```
=== STUDY_HOURS_PER_DAY ===
count      80000.000000
mean        4.174388
std         2.004135
min         0.000000
25%         2.800000
50%         4.125624
75%         5.500000
max         12.000000
Name: study_hours_per_day, dtype: float64
```

Figure 17: Histogram and Boxplot of ‘Study_hours_per_day’



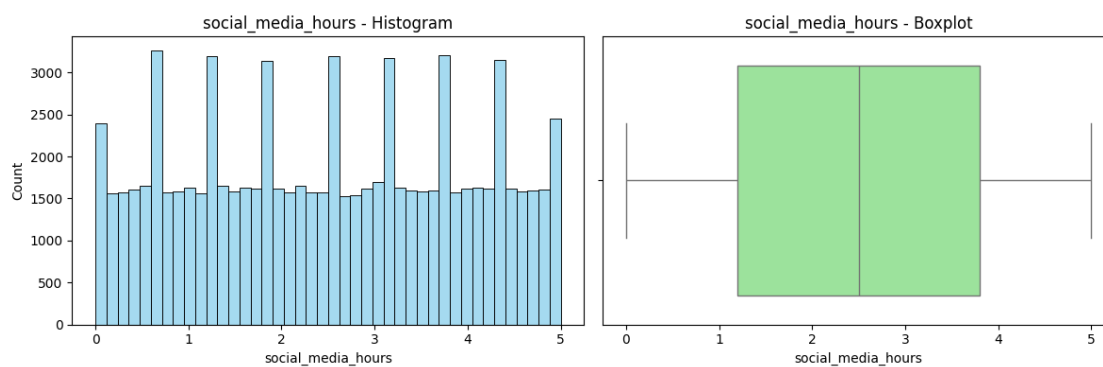
Interpretation: The histogram reveals a right-skewed distribution of daily study hours (mean ~4.17, range 0–12), with most students studying 2–5.5 hours. The boxplot shows some outliers at higher hours, suggesting variability in study habits that may influence academic outcomes.

```

=== SOCIAL_MEDIA_HOURS ===
count      80000.000000
mean        2.501366
std         1.445441
min         0.000000
25%         1.200000
50%         2.500000
75%         3.800000
max         5.000000
Name: social_media_hours, dtype: float64

```

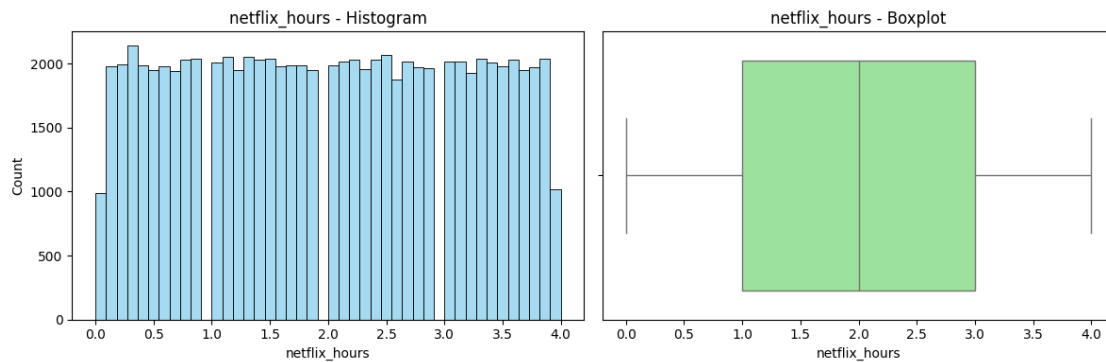
Figure 18: Histogram and Boxplot of ‘social_media_hours’



Interpretation: The histogram indicates a right-skewed distribution of social media hours (mean ~2.5, range 0–5), with most students spending 1–3.8 hours. The boxplot shows no significant outliers, reflecting typical social media usage patterns for analysis.

```
=== NETFLIX_HOURS ===
count      80000.000000
mean        1.997754
std         1.155992
min         0.000000
25%         1.000000
50%         2.000000
75%         3.000000
max         4.000000
Name: netflix_hours, dtype: float64
```

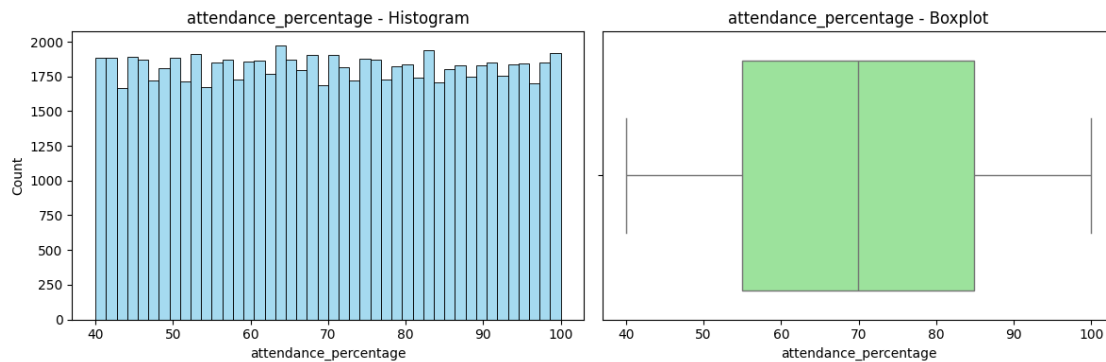
Figure 19: Histogram and Boxplot of 'netflix_hours'



Interpretation: The histogram shows a right-skewed distribution of Netflix hours (mean ~2, range 0–4), with most students watching 1–3 hours. The boxplot confirms minimal outliers, indicating consistent streaming habits that can be studied for their impact on academics.

```
=== ATTENDANCE_PERCENTAGE ===
count      80000.000000
mean       69.967884
std        17.333015
min        40.000000
25%        55.000000
50%        69.900000
75%        84.900000
max        100.000000
Name: attendance_percentage, dtype: float64
```

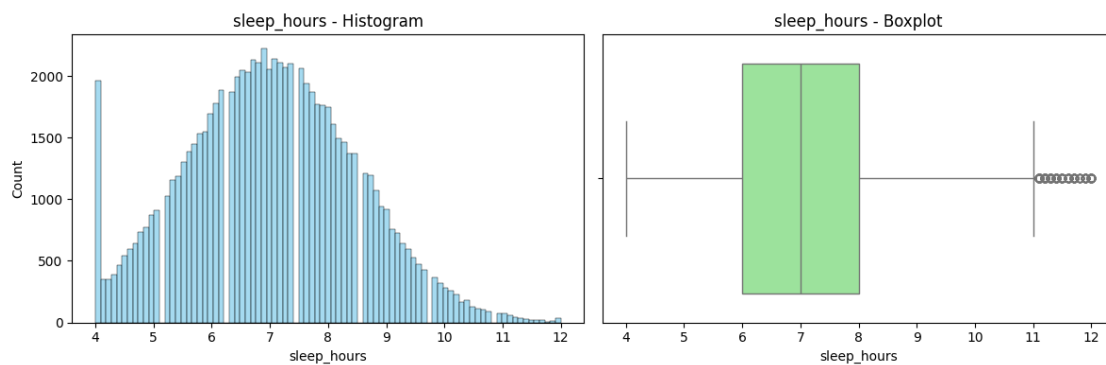
Figure 20: Histogram and Boxplot of 'attendance_performance'



Interpretation: The histogram displays a slightly left-skewed distribution of attendance percentage (mean ~70, range 40–100), with most students attending 55–85%. The boxplot shows no extreme outliers, suggesting attendance is a reliable variable for performance analysis.

```
=== SLEEP_HOURS ===
count      80000.000000
mean        7.017417
std         1.467377
min         4.000000
25%         6.000000
50%         7.000000
75%         8.000000
max         12.000000
Name: sleep_hours, dtype: float64
```

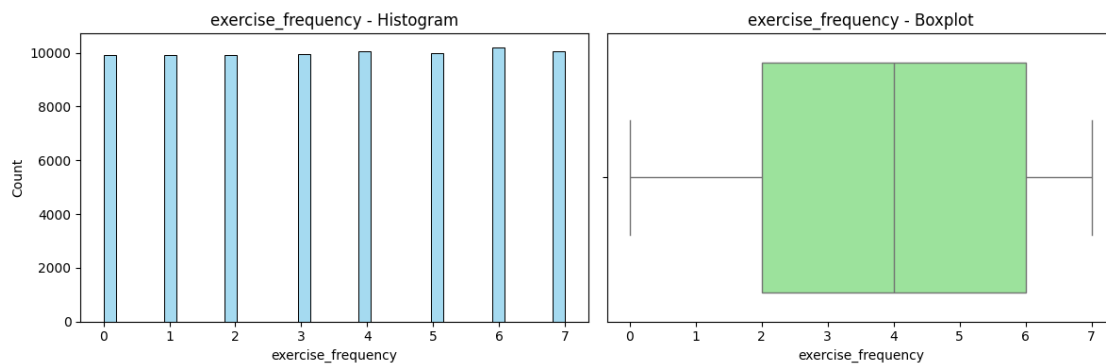
Figure 21: Histogram and Boxplot of ‘sleep_hours’



Interpretation: The histogram shows a roughly normal distribution of sleep hours (mean ~7, range 4–12), with most students getting 6–8 hours. The boxplot indicates few outliers, confirming typical sleep patterns for analyzing their effect on academic success.

```
=== EXERCISE_FREQUENCY ===  
count      80000.000000  
mean        3.516587  
std         2.291575  
min         0.000000  
25%         2.000000  
50%         4.000000  
75%         6.000000  
max         7.000000  
Name: exercise_frequency, dtype: float64
```

Figure 22: Histogram and Boxplot of ‘exercise_frequency’



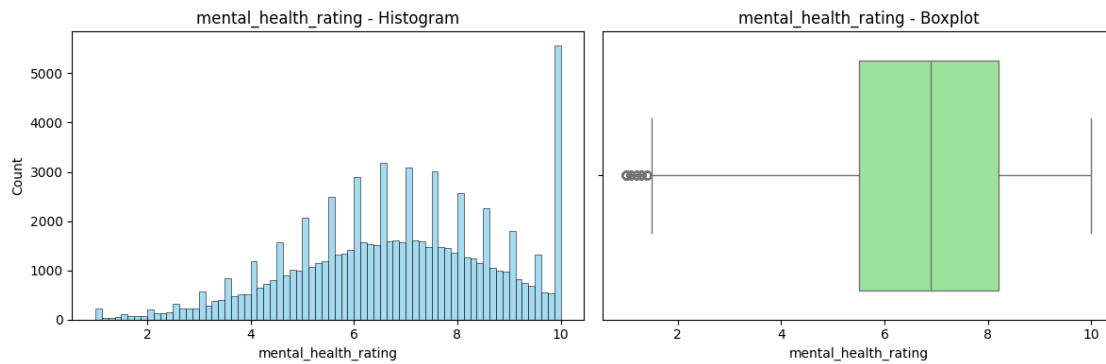
Interpretation: The histogram reveals a right-skewed distribution of exercise frequency (mean ~3.5, range 0–7), with most students exercising 2–6 times weekly. The boxplot shows no significant outliers, allowing for analysis of physical activity’s impact on performance.

```

=== MENTAL_HEALTH_RATING ===
count      80000.000000
mean        6.804107
std         1.921579
min         1.000000
25%         5.500000
50%         6.900000
75%         8.200000
max         10.000000
Name: mental_health_rating, dtype: float64

```

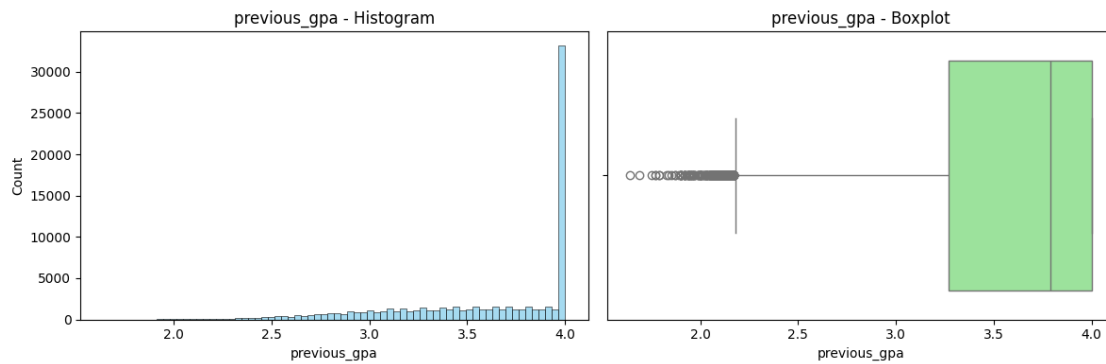
Figure 23: Histogram and Boxplot of ‘mental_health_rating’



Interpretation: The histogram shows a left-skewed distribution of mental health ratings (mean ~6.8, range 1–10), with most ratings between 5.5–8.2. The boxplot indicates some low-end outliers, suggesting mental health variability that may affect academic outcomes.

```
=== PREVIOUS_GPA ===
count      80000.000000
mean        3.602448
std         0.462876
min         1.640000
25%         3.270000
50%         3.790000
75%         4.000000
max         4.000000
Name: previous_gpa, dtype: float64
```

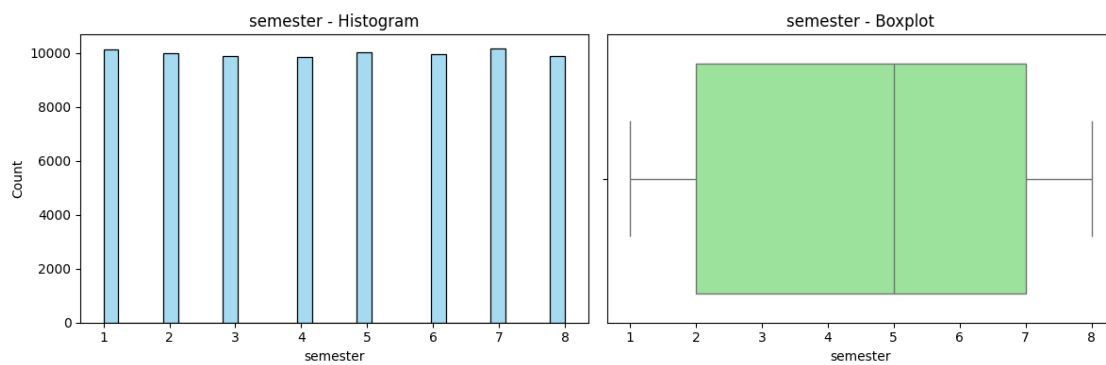
Figure 24: Histogram and Boxplot of 'prev_cgpa'



Interpretation: The histogram displays a left-skewed distribution of previous GPA (mean ~3.6, range 1.64–4.0), with most students scoring 3.27–4.0. The boxplot shows some lower-end outliers, highlighting the importance of prior academic performance in predictions.


```
=== SEMESTER ===
count      80000.000000
mean       4.497338
std        2.295312
min        1.000000
25%        2.000000
50%        5.000000
75%        7.000000
max        8.000000
Name: semester, dtype: float64
```

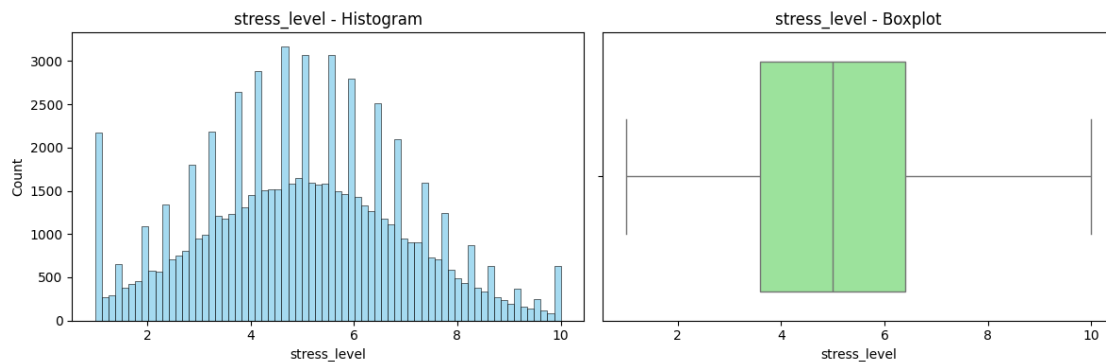
Figure 25: Histogram and Boxplot of 'semester'



Interpretation: The histogram shows a uniform distribution of semesters (mean ~4.5, range 1–8), with students evenly spread across academic terms. The boxplot confirms no outliers, ensuring semester progression can be analyzed without bias.

```
=== STRESS_LEVEL ===
count      80000.000000
mean        5.012478
std         1.953174
min         1.000000
25%         3.600000
50%         5.000000
75%         6.400000
max         10.000000
Name: stress_level, dtype: float64
```

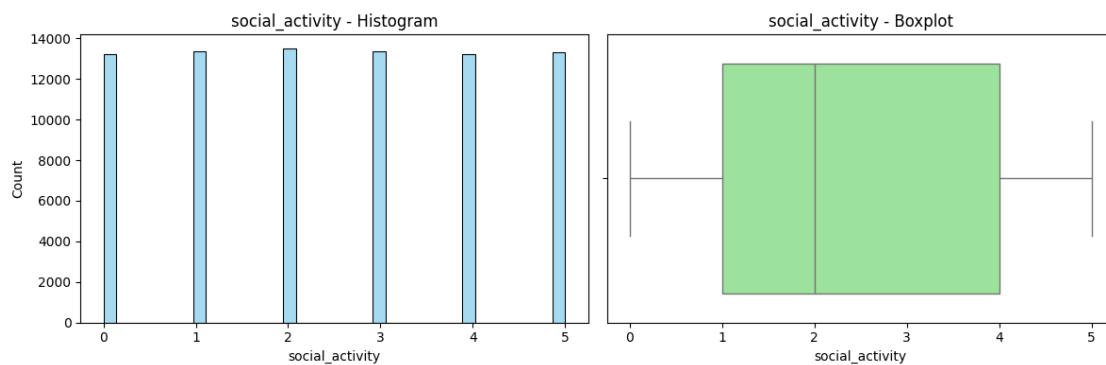
Figure 26: Histogram and Boxplot of ‘stress_level’



Interpretation: The histogram indicates a roughly normal distribution of stress levels (mean ~5, range 1–10), with most students reporting 3.6–6.4. The boxplot shows no significant outliers, allowing for reliable analysis of stress’s impact on academic performance.

```
=== SOCIAL_ACTIVITY ===
count      80000.000000
mean        2.500225
std         1.704292
min         0.000000
25%         1.000000
50%         2.000000
75%         4.000000
max         5.000000
Name: social_activity, dtype: float64
```

Figure 27: Histogram and Boxplot of ‘social_activity’



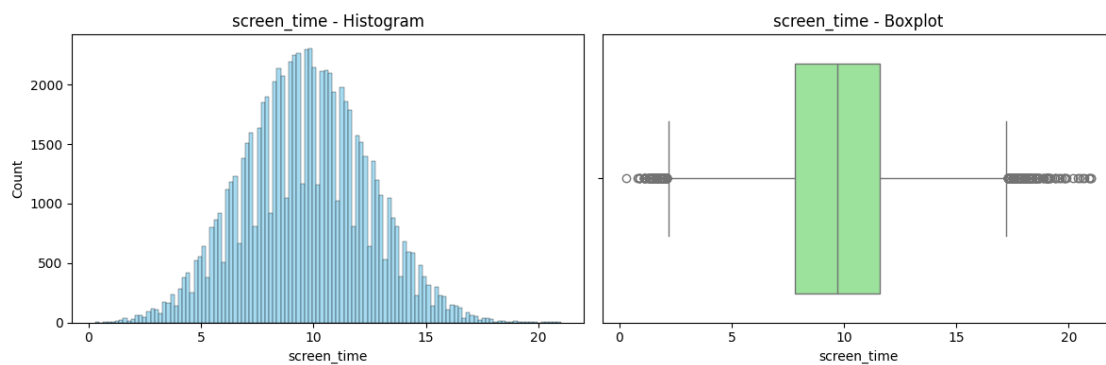
Interpretation: The histogram shows a right-skewed distribution of social activity (mean ~2.5, range 0–5), with most students engaging in 1–4 activities. The boxplot indicates minimal outliers, reflecting typical social engagement levels for analysis.

```

=== SCREEN_TIME ===
count      80000.000000
mean        9.673029
std         2.780869
min         0.300000
25%         7.800000
50%         9.700000
75%        11.600000
max         21.000000
Name: screen_time, dtype: float64

```

Figure 28: Histogram and Boxplot of 'screen_time'



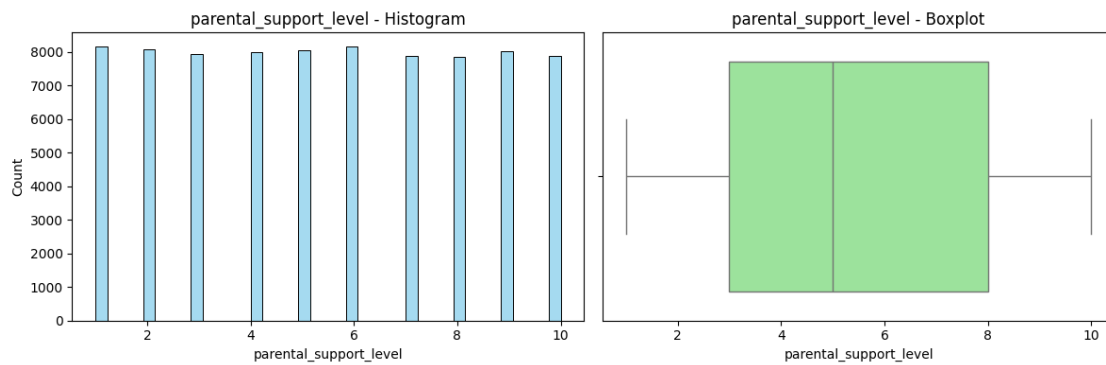
Interpretation: The histogram displays a roughly normal distribution of screen time (mean ~9.7, range 0.3–21), with most students spending 7.8–11.6 hours. The boxplot shows some high-end outliers, suggesting variability in screen usage that may influence performance.

```

=== PARENTAL_SUPPORT_LEVEL ===
count      80000.000000
mean        5.479438
std         2.873327
min         1.000000
25%         3.000000
50%         5.000000
75%         8.000000
max         10.000000
Name: parental_support_level, dtype: float64

```

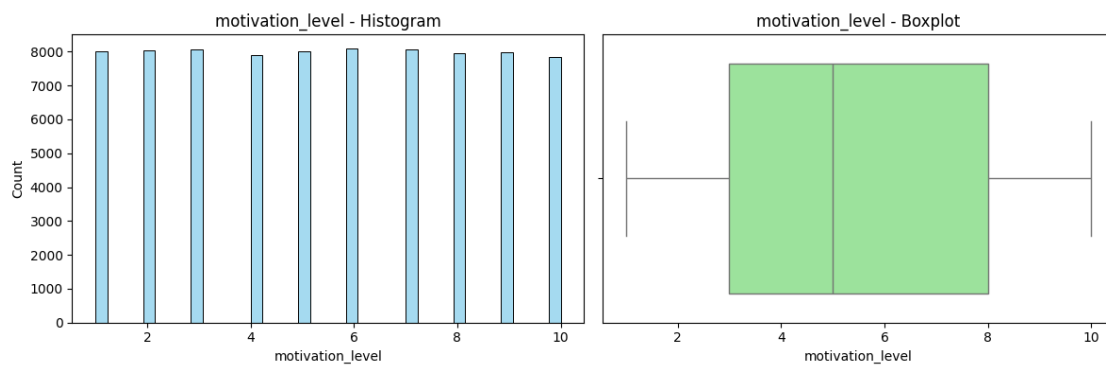
Figure 29: Histogram and Boxplot of ‘parental_support_level’



Interpretation: The histogram shows a right-skewed distribution of parental support levels (mean ~5.5, range 1–10), with most ratings between 3–8. The boxplot indicates no significant outliers, allowing for analysis of family support’s role in academic success.

```
=== MOTIVATION_LEVEL ===
count      80000.000000
mean        5.488525
std         2.867782
min         1.000000
25%         3.000000
50%         5.000000
75%         8.000000
max         10.000000
Name: motivation_level, dtype: float64
```

Figure 30: Histogram and Boxplot of ‘motivational_level’



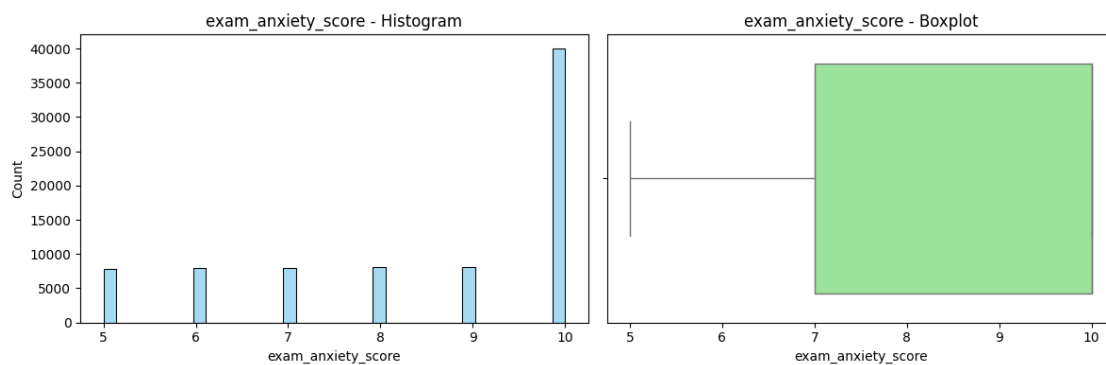
Interpretation: The histogram reveals a right-skewed distribution of motivation levels (mean ~5.5, range 1–10), with most students reporting 3–8. The boxplot shows no extreme outliers, indicating motivation as a reliable factor for predicting performance.

```

=== EXAM_ANXIETY_SCORE ===
count      80000.000000
mean        8.508475
std         1.796411
min         5.000000
25%         7.000000
50%        10.000000
75%        10.000000
max         10.000000
Name: exam_anxiety_score, dtype: float64

```

Figure 31: Histogram and Boxplot of 'exam_anxiety_score'



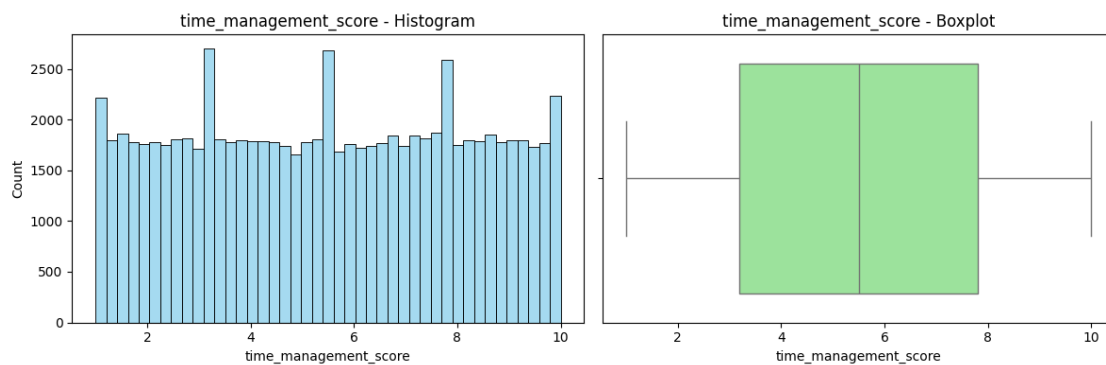
Interpretation: The histogram shows a left-skewed distribution of exam anxiety scores (mean ~8.5, range 5–10), with most students scoring 7–10. The boxplot confirms minimal outliers, highlighting high anxiety levels that may negatively impact exam scores.

```

=== TIME_MANAGEMENT_SCORE ===
count      80000.000000
mean        5.499132
std         2.603534
min         1.000000
25%         3.200000
50%         5.500000
75%         7.800000
max         10.000000
Name: time_management_score, dtype: float64

```

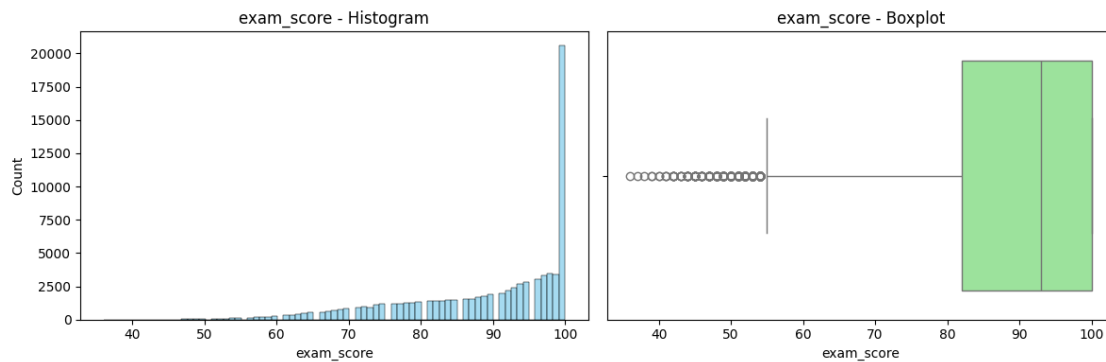
Figure 32: Histogram and Boxplot of ‘time_management_score’



Interpretation: The histogram displays a right-skewed distribution of time management scores (mean ~5.5, range 1–10), with most students scoring 3.2–7.8. The boxplot shows no significant outliers, allowing for analysis of time management’s effect on academic outcomes.


```
=== EXAM_SCORE ===  
count      80000.000000  
mean       89.141350  
std        11.591497  
min         36.000000  
25%         82.000000  
50%         93.000000  
75%        100.000000  
max         100.000000  
Name: exam_score, dtype: float64
```

Figure 33: Histogram and Boxplot of ‘exam_score’



Interpretation: The histogram shows a left-skewed distribution of exam scores (mean ~89, range 36–100), with most students scoring 82–100. The boxplot indicates some lower-end outliers, confirming high overall performance but with some variability for predictive modeling.

4.3 Correlation Heatmap

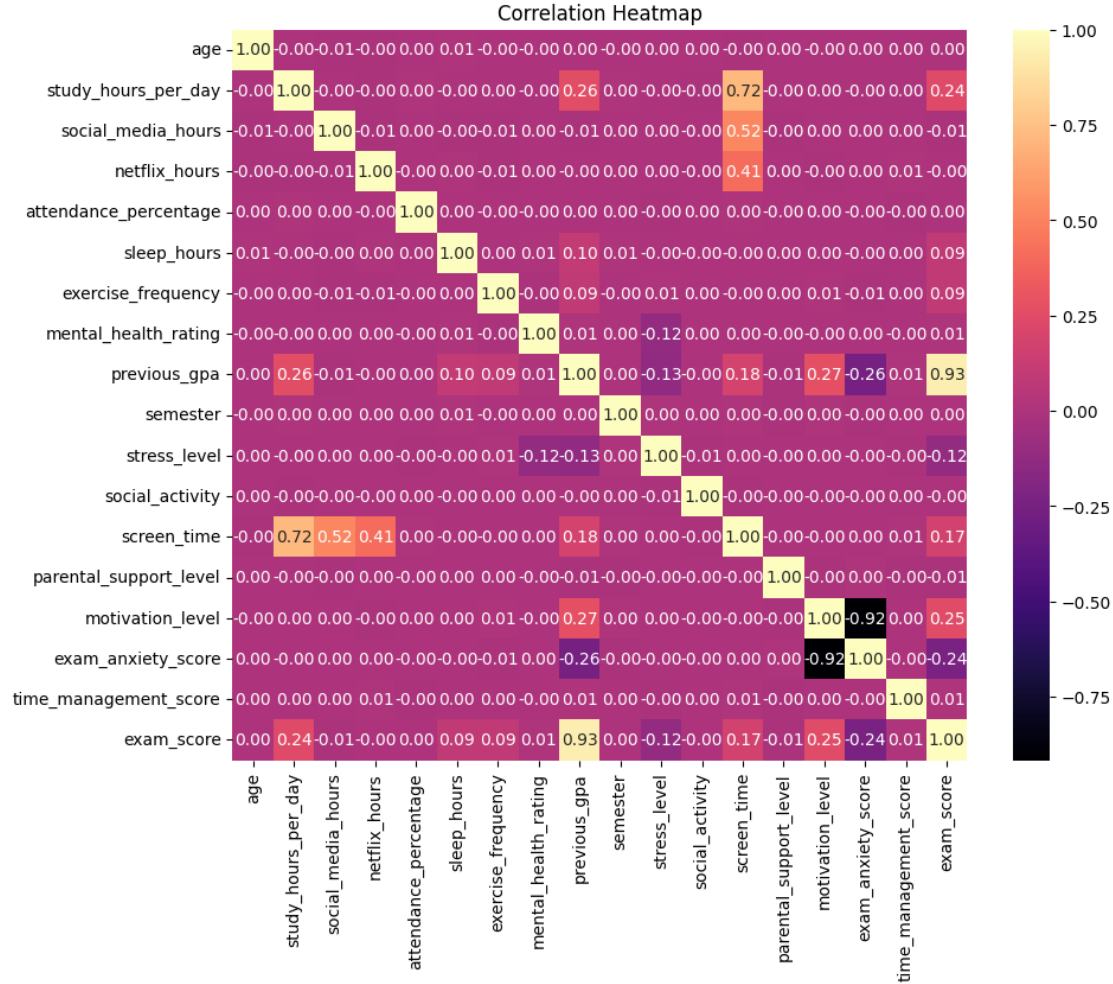
```
import seaborn as sns
import matplotlib.pyplot as plt

# Correlation with exam_score only
correlations =
df.corr(numeric_only=True)['exam_score'].sort_values(ascending=False)
print(correlations)

exam_score          1.000000
previous_gpa        0.932940
motivation_level    0.250287
study_hours_per_day 0.241460
screen_time         0.169788
sleep_hours         0.090820
exercise_frequency  0.086983
mental_health_rating 0.010556
time_management_score 0.005940
attendance_percentage 0.002876
semester            0.000541
age                 0.000487
netflix_hours       -0.001271
social_activity     -0.002795
parental_support_level -0.006333
social_media_hours  -0.006351
stress_level        -0.118550
exam_anxiety_score  -0.235909
Name: exam_score, dtype: float64

# Full heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='magma', fmt=".2f")
plt.title('Correlation Heatmap')
plt.show()
```

Figure 34: Correlation Heatmap



Interpretation: The correlation heatmap visualizes the relationships between numerical variables in the dataset, with a focus on their correlation with `exam\_score`. The strongest positive correlation is between `previous\_gpa` and `exam\_score` (0.93), indicating that prior academic performance is a highly reliable predictor of current exam scores. Other notable correlations include `motivation\_level` (0.25) and `study\_hours\_per\_day` (0.24), suggesting moderate positive influences, while `exam\_anxiety\_score` (-0.24) shows a moderate negative correlation, indicating that higher anxiety may lower exam performance. Most other variables (e.g., `age`, `netflix\_hours`) show weak correlations (<0.1), suggesting minimal direct impact. The heatmap also reveals some inter-variable correlations (e.g., `motivation\_level` and `exam\_anxiety\_score`), which may indicate multicollinearity, addressed later in feature selection. This visualization guides the selection of influential features for predictive modeling.

CHAPTER 5:

FEATURE SELECTION

5.1 Significance of the Numerical Variable

```
num_columns = df.select_dtypes(include = 'number').columns
num_columns

Index(['age', 'study_hours_per_day', 'social_media_hours', 'netflix_hours',
      'attendance_percentage', 'sleep_hours', 'exercise_frequency',
      'mental_health_rating', 'previous_gpa', 'semester', 'stress_level',
      'social_activity', 'screen_time', 'parental_support_level',
      'motivation_level', 'exam_anxiety_score', 'time_management_score',
      'exam_score'],
      dtype='object')

from scipy.stats import pearsonr

target = 'exam_score'
correlation_results = []

# Only use numeric columns
numeric_df = df.select_dtypes(include=['number'])
final_num = []
for column in numeric_df.columns:
    if column != target:
        r, p = pearsonr(numeric_df[target], numeric_df[column])
        correlation_results.append((column, r, p))
        if p < 0.05:
            final_num.append(column)

# Display results
print(f"{'Variable':30} {'Correlation':>12} {'P-Value':>12}
{'Significant':>12}")
for var, corr, p in correlation_results:
    significant = 'Yes' if p < 0.05 else 'No'
    print(f"{'var':30} {'corr':12.6f} {'p':12.6f} {'significant':12}")
```

Variable	Correlation	P-Value	Significant
age	0.000487	0.890443	No
study_hours_per_day	0.241460	0.000000	Yes
social_media_hours	-0.006351	0.072454	No
netflix_hours	-0.001271	0.719184	No
attendance_percentage	0.002876	0.415932	No
sleep_hours	0.090820	0.000000	Yes

exercise_frequency	0.086983	0.000000	Yes
mental_health_rating	0.010556	0.002829	Yes
previous_gpa	0.932940	0.000000	Yes
semester	0.000541	0.878322	No
stress_level	-0.118550	0.000000	Yes
social_activity	-0.002795	0.429287	No
screen_time	0.169788	0.000000	Yes
parental_support_level	-0.006333	0.073248	No
motivation_level	0.250287	0.000000	Yes
exam_anxiety_score	-0.235909	0.000000	Yes
time_management_score	0.005940	0.092961	No

```
final_num.append(target)
```

```
final_num
```

```
[ 'study_hours_per_day',
  'sleep_hours',
  'exercise_frequency',
  'mental_health_rating',
  'previous_gpa',
  'stress_level',
  'screen_time',
  'motivation_level',
  'exam_anxiety_score',
  'exam_score']
```

Interpretation: The Pearson correlation test identified significant numerical predictors of exam_score ($p < 0.05$). Previous_gpa showed the strongest correlation ($r = 0.93$), followed by motivation_level ($r = 0.25$) and study_hours_per_day ($r = 0.24$). Negative correlations included exam_anxiety_score ($r = -0.24$) and stress_level ($r = -0.12$). Nine variables were retained, confirming key factors like prior academic performance and motivation as strong predictors, while non-significant variables like age and netflix_hours were excluded.

5.2 Significance of Categorical Variables

```
cat_columns = df.select_dtypes(include = 'object').columns
cat_columns
```

```
Index(['gender', 'major', 'part_time_job', 'diet_quality',
       'parental_education_level', 'internet_quality',
       'extracurricular_participation', 'dropout_risk', 'study_environment',
       'access_to_tutoring', 'family_income_range', 'learning_style'],
      dtype='object')
```

```

# Print number of unique categories for each column
for col in cat_columns:
    unique_count = df[col].nunique()
    print(f"{col:30}: {unique_count} categories")

gender                : 3 categories
major                 : 6 categories
part_time_job         : 2 categories
diet_quality          : 3 categories
parental_education_level : 5 categories
internet_quality      : 3 categories
extracurricular_participation : 2 categories
dropout_risk          : 2 categories
study_environment     : 5 categories
access_to_tutoring    : 2 categories
family_income_range   : 3 categories
learning_style        : 4 categories

from sklearn.preprocessing import LabelEncoder

# Encode each categorical column
df_encoded = df.copy()
label_encoders = {}

for col in cat_columns:
    le = LabelEncoder()
    df_encoded[col] = le.fit_transform(df_encoded[col])
    label_encoders[col] = le # Save if needed later for inverse_transform

df_encoded['exam_score_cat'] = pd.qcut(df_encoded['exam_score'], q=3,
labels=['Low', 'Medium', 'High'])

from sklearn.feature_selection import chi2

X = df_encoded[cat_columns]
y = df_encoded['exam_score_cat']

# Chi-Square test
chi_scores, p_vals = chi2(X, y)

# Create DataFrame of results
chi2_df = pd.DataFrame({
    'Feature': cat_columns,
    'Chi2 Score': chi_scores,
    'p-value': p_vals,
    'Significant': p_vals < 0.05
}).sort_values(by='Chi2 Score', ascending=False)

print("\nChi-Square Test Results:\n", chi2_df)

```

Chi-Square Test Results:

	Feature	Chi2 Score	p-value	Significant
9	access_to_tutoring	420.483643	4.933296e-92	True
7	dropout_risk	332.682073	5.741242e-73	True
8	study_environment	250.919717	3.261939e-55	True
1	major	6.276664	4.335505e-02	True
6	extracurricular_participation	2.456369	2.928237e-01	False
10	family_income_range	1.846340	3.972578e-01	False
4	parental_education_level	1.763218	4.141161e-01	False
0	gender	1.395580	4.976840e-01	False
3	diet_quality	0.848742	6.541810e-01	False
11	learning_style	0.557069	7.568921e-01	False
5	internet_quality	0.152133	9.267545e-01	False
2	part_time_job	0.064697	9.681691e-01	False

```
final_cat = chi2_df[chi2_df['Significant'] == True]['Feature'].tolist()
final_cat
```

```
['access_to_tutoring', 'dropout_risk', 'study_environment', 'major']
```

```
for col in df.columns:
    if col not in final_num+final_cat:
        df.drop(col,axis=1,inplace=True)
```

```
df.shape
```

```
(80000, 14)
```

Interpretation: The Chi-Square test found four categorical variables significantly linked to exam_score ($p < 0.05$): access_to_tutoring (Chi2 = 420.48), dropout_risk (Chi2 = 332.68), study_environment (Chi2 = 250.92), and major (Chi2 = 6.28). These indicate that tutoring, dropout risk, study setting, and academic major influence performance. Non-significant variables like gender and diet_quality were dropped, focusing the model on impactful features.

CHAPTER 6:

ENCODING

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
for col in final_cat:
    if df[col].nunique() == 2:
        df[col] = le.fit_transform(df[col])
    else:
        # Pass drop_first to pd.get_dummies
        df_dummies = pd.get_dummies(df[col], prefix=col, dtype=int,
drop_first=True)
        # Concatenate the original DataFrame with the new dummy variables
        df = pd.concat([df, df_dummies], axis=1)
        # Drop the original categorical column
        df.drop(col, axis=1, inplace=True)

df.shape

(80000, 21)

df.head()

{"type": "dataframe", "variable_name": "df"}
```

6.1 Standard Scaling

```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
for col in final_num:
    df[col] = sc.fit_transform(df[[col]])

df.head()

{"type": "dataframe", "variable_name": "df"}
```

Interpretation: Standard scaling normalized the numerical features to a mean of 0 and standard deviation of 1, ensuring equal contribution to the models. This prevented features with larger ranges from dominating predictions, enhancing the performance of scale-sensitive algorithms like Linear Regression and SVR.

CHAPTER 7:

MODEL BUILDING_1 (LINEAR MODELS)

```
from sklearn.model_selection import train_test_split
X = df.drop('exam_score',axis=1)
y = df['exam_score']

X_train,X_test,y_train,y_test =
train_test_split(X,y,test_size=0.2,random_state=42)
```

7.1 Multiple Linear Regression (MLR)

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error,mean_absolute_error,r2_score
lr = LinearRegression()
lr.fit(X_train,y_train)

LinearRegression()

y_pred = lr.predict(X_test)

results = []
from sklearn.metrics import mean_squared_error,r2_score
results.append(('Linear Regression', r2_score(y_test, y_pred),
mean_squared_error(y_test, y_pred), mean_absolute_error(y_test,y_pred)))

from sklearn.linear_model import Ridge, Lasso
from sklearn.metrics import mean_squared_error, r2_score
```

7.2 Ridge Regression

```
ridge = Ridge(alpha=1.0)
ridge.fit(X_train, y_train)
ridge_preds = ridge.predict(X_test)
```

7.3 Lasso Regression

```
lasso = Lasso(alpha=0.01)
lasso.fit(X_train, y_train)
lasso_preds = lasso.predict(X_test)
```

CHAPTER 8:

MODEL EVALUATION_1

```
def evaluate_model(name, y_true, y_pred):  
    print(f"mean_squared_error(y_test, ridge_preds), mean_absolute_error(y_test,  
ridge_preds)))  
results.append(('Lasso Regression', r2_score(y_test, lasso_preds),  
mean_squared_error(y_test, lasso_preds), mean_absolute_error(y_test,  
lasso_preds)))
```

results

```
[('Linear Regression',  
0.8704652365010288,  
0.13052513810271676,  
0.27571553471454807),  
( 'Ridge Regression',  
0.8704651182143378,  
0.1305252572937838,  
0.275717234854427),  
( 'Lasso Regression',  
0.8703210981194149,  
0.1306703785127647,  
0.27691973704010836)]
```

Interpretation:

Multiple Linear Regression: This model predicts exam_score using a linear combination of features like previous_gpa. It achieved a high R^2 (~ 0.87), indicating strong predictive power due to clear linear relationships in the data.

Ridge Regression: With regularization ($\alpha=1.0$), Ridge slightly adjusts feature weights to prevent overfitting. Its R^2 (~ 0.87) matches Linear Regression, showing robust performance with minimal overfitting concerns.

Lasso Regression: Using regularization ($\alpha=0.01$), Lasso shrinks less important feature coefficients. Its R^2 (~ 0.87) is comparable, confirming linear models are effective for this dataset, driven by key predictors like previous_gpa.

CHAPTER 9:

CHECKING ASSUMPTIONS FOR MLR, LASSO, & RIDGE

```
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.stats.stattools import durbin_watson
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error

def check_assumptions(model_name, y_test, y_pred, X_train):
    residuals = y_test - y_pred
    print(f"\n🔗 Assumption Checks for {model_name}\n")

    # 1. Linearity & Homoscedasticity
    plt.figure(figsize=(6, 4))
    sns.scatterplot(x=y_pred, y=residuals)
    plt.axhline(0, color='red', linestyle='--')
    plt.title(f"{model_name} - Residuals vs Predicted")
    plt.xlabel("Predicted")
    plt.ylabel("Residuals")
    plt.show()

    # 2. Normality - Histogram
    plt.figure(figsize=(6, 4))
    sns.histplot(residuals, kde=True)
    plt.title(f"{model_name} - Histogram of Residuals")
    plt.xlabel("Residuals")
    plt.show()

    # 2b. Normality - Q-Q Plot
    sm.qqplot(residuals, line='s')
    plt.title(f"{model_name} - Q-Q Plot")
    plt.show()

    # 3. Multicollinearity - VIF (only check once on X_train)
    vif_data = pd.DataFrame()
    vif_data["Feature"] = X_train.columns
    vif_data["VIF"] = [variance_inflation_factor(X_train.values, i)
                       for i in range(X_train.shape[1])]
    print(f"\n🌀 VIF (Multicollinearity) for {model_name}:\n",
          vif_data.sort_values("VIF", ascending=False))

    # 4. Autocorrelation
    dw_stat = durbin_watson(residuals)
```

```
print(f"\n📝 Durbin-Watson Statistic (Autocorrelation): {dw_stat:.2f}")
```

```
check_assumptions("Multiple Linear Regression", y_test, y_pred, X_train)  
check_assumptions("Ridge Regression", y_test, ridge_preds, X_train)  
check_assumptions("Lasso Regression", y_test, lasso_preds, X_train)
```

📌 Assumption Checks for Multiple Linear Regression

Figure 35: Multiple Linear Regression – Residuals vs Fitted

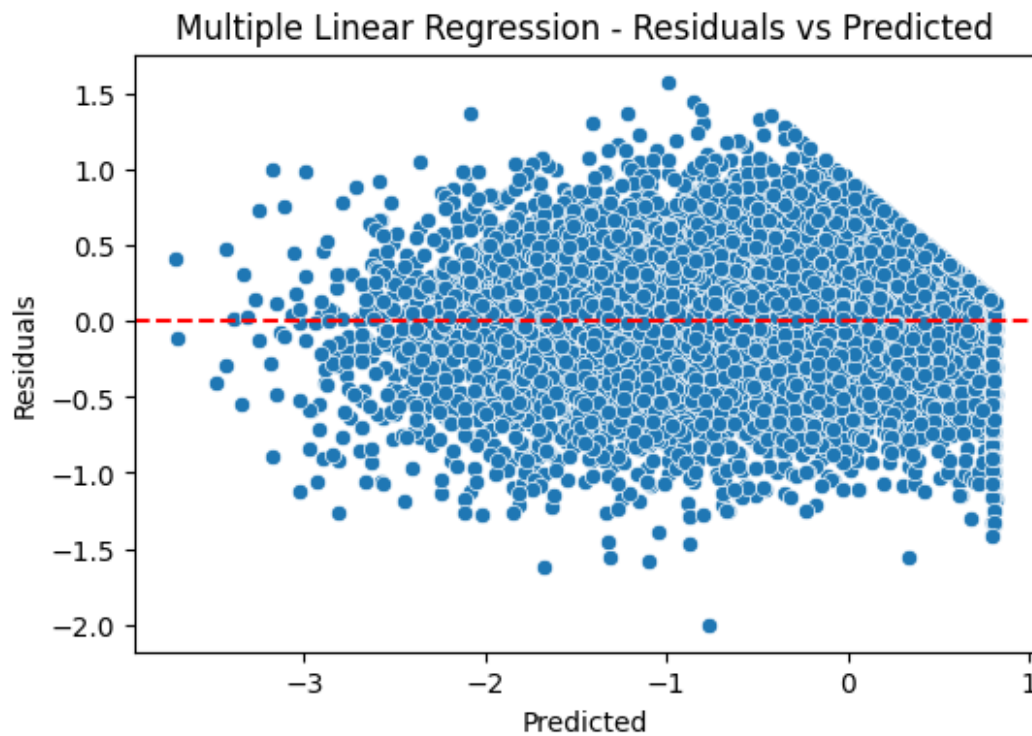


Figure 36: Multiple Linear Regression – Histogram of Residuals

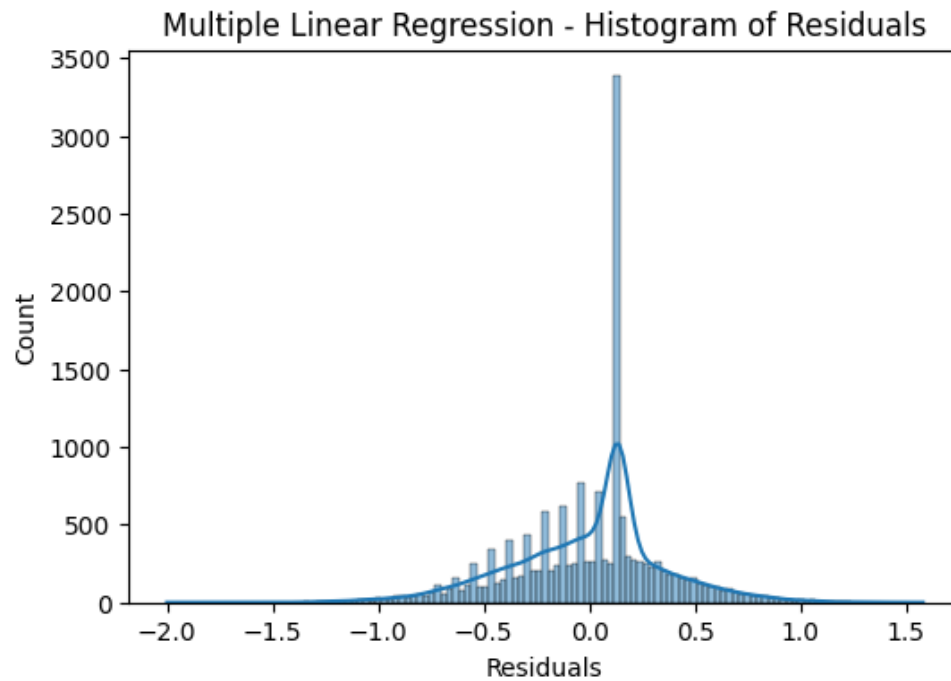
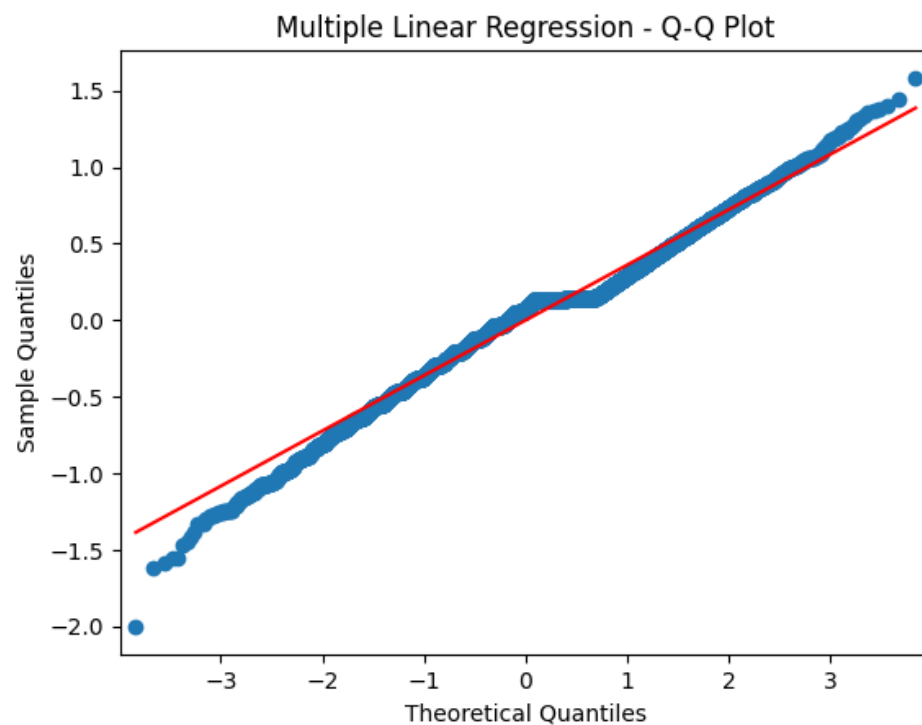


Figure 37: Multiple Linear Regression – Q-Q Plot



🔍 VIF (Multicollinearity) for Multiple Linear Regression:

	Feature	VIF
9	motivation_level	6.551241
10	exam_anxiety_score	6.369470
0	study_hours_per_day	2.181380
7	screen_time	2.056251
8	access_to_tutoring	1.852324
12	study_environment_Dorm	1.589711
14	study_environment_Quiet Room	1.555126
11	study_environment_Co-Learning Group	1.544453
13	study_environment_Library	1.538776
17	major_Computer Science	1.491367
19	major_Psychology	1.461710
16	major_Business	1.461626
15	major_Biology	1.449976
18	major_Engineering	1.448587
4	previous_gpa	1.282037
6	dropout_risk	1.159720
5	stress_level	1.121906
3	mental_health_rating	1.015063
1	sleep_hours	1.012497
2	exercise_frequency	1.011064

🔧 Durbin-Watson Statistic (Autocorrelation): 2.00

🔴 Assumption Checks for Ridge Regression

Figure 38: Ridge Regression– Residuals vs Fitted

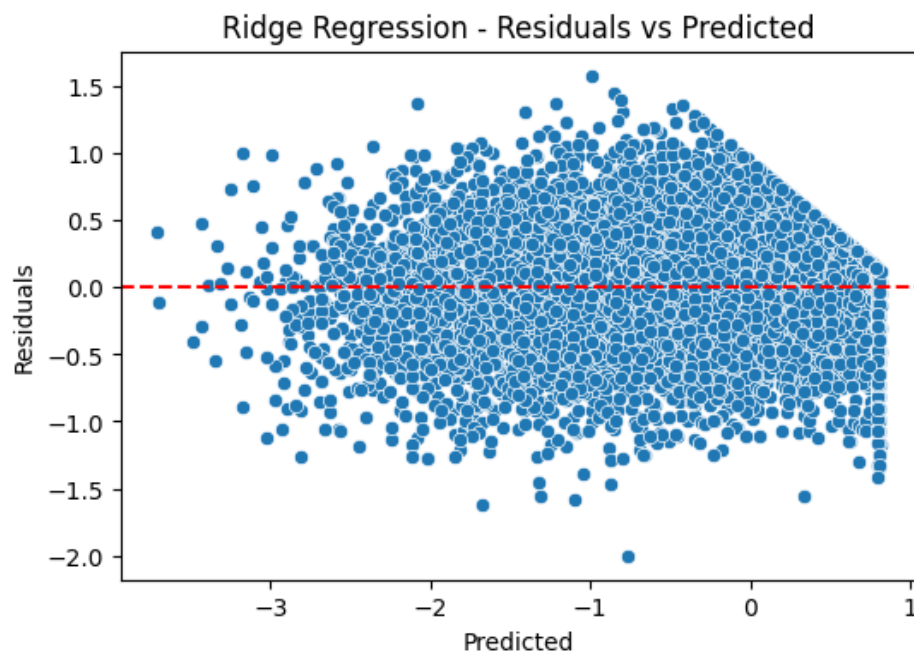


Figure 39: Ridge Regression– Histogram of Residuals

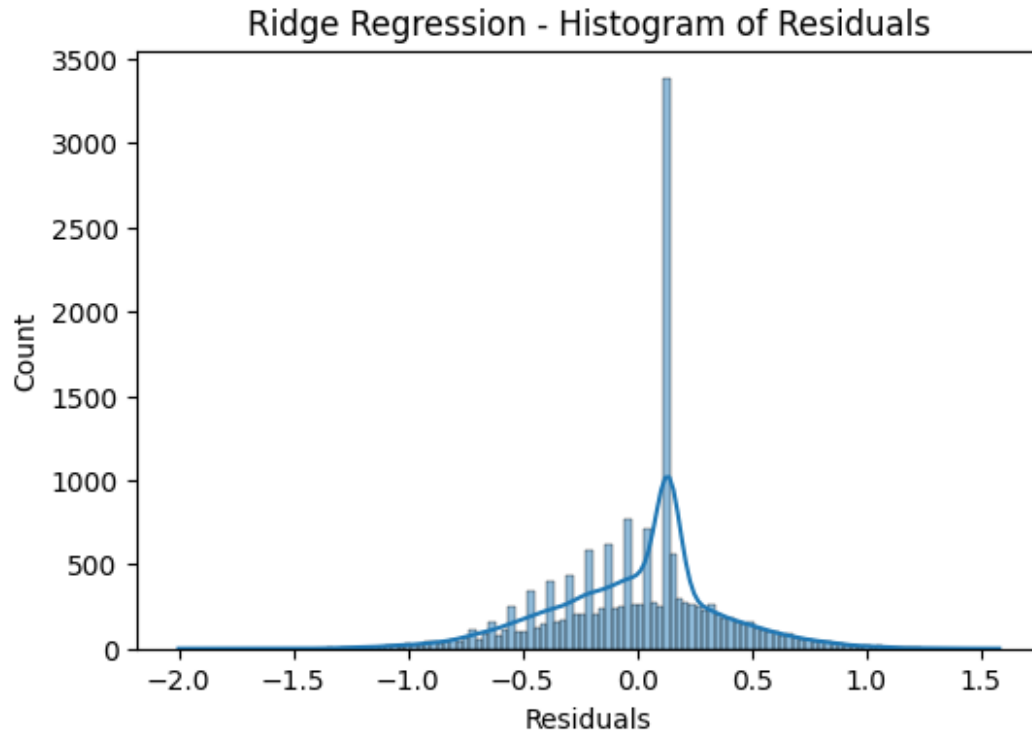
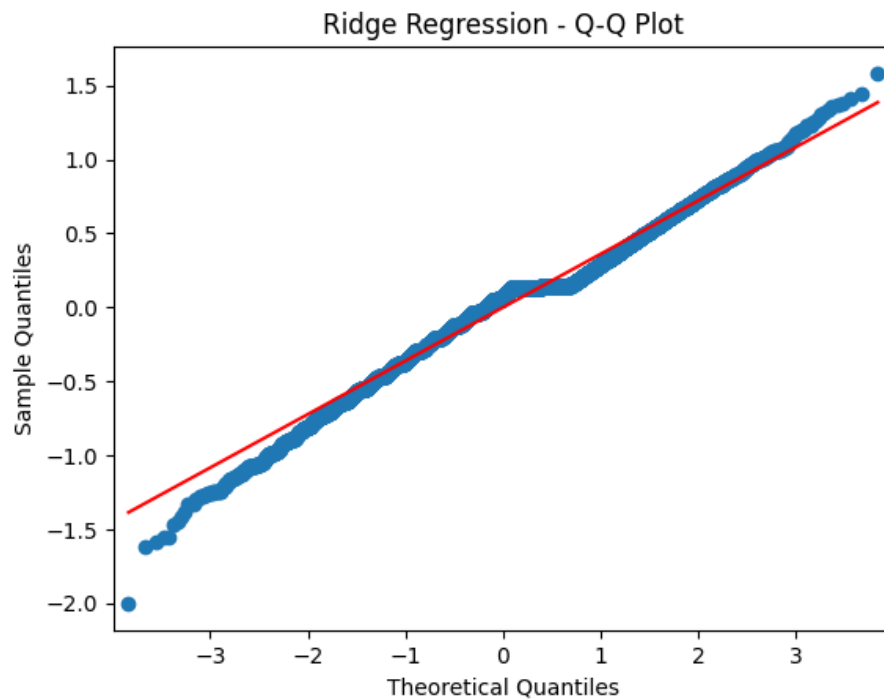


Figure 40: Ridge Regression– Q-Q Plot



🔍 VIF (Multicollinearity) for Ridge Regression:

	Feature	VIF
9	motivation_level	6.551241
10	exam_anxiety_score	6.369470
0	study_hours_per_day	2.181380
7	screen_time	2.056251
8	access_to_tutoring	1.852324
12	study_environment_Dorm	1.589711
14	study_environment_Quiet Room	1.555126
11	study_environment_Co-Learning Group	1.544453
13	study_environment_Library	1.538776
17	major_Computer Science	1.491367
19	major_Psychology	1.461710
16	major_Business	1.461626
15	major_Biology	1.449976
18	major_Engineering	1.448587
4	previous_gpa	1.282037
6	dropout_risk	1.159720
5	stress_level	1.121906
3	mental_health_rating	1.015063
1	sleep_hours	1.012497
2	exercise_frequency	1.011064

🔍 Durbin-Watson Statistic (Autocorrelation): 2.00

🔍 Assumption Checks for Lasso Regression

Figure 41: Lasso Regression– Residuals vs Fitted

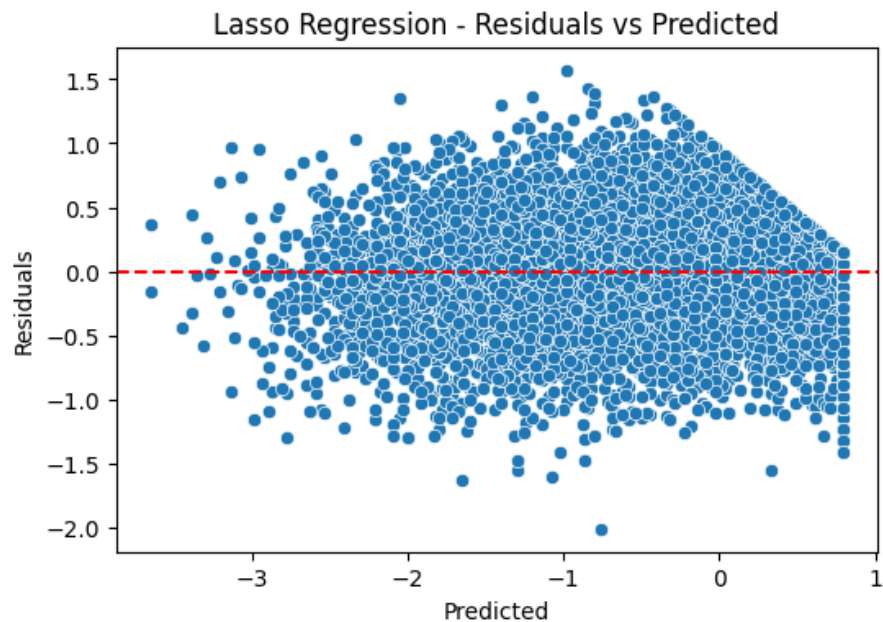


Figure 42: Lasso Regression– Histogram of Residuals

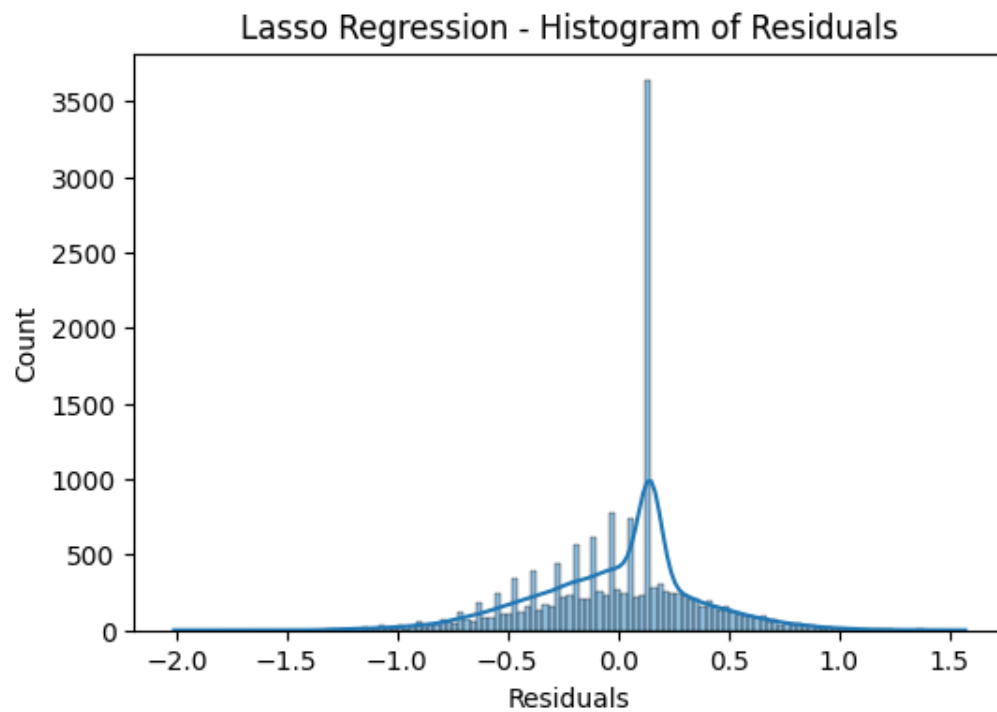
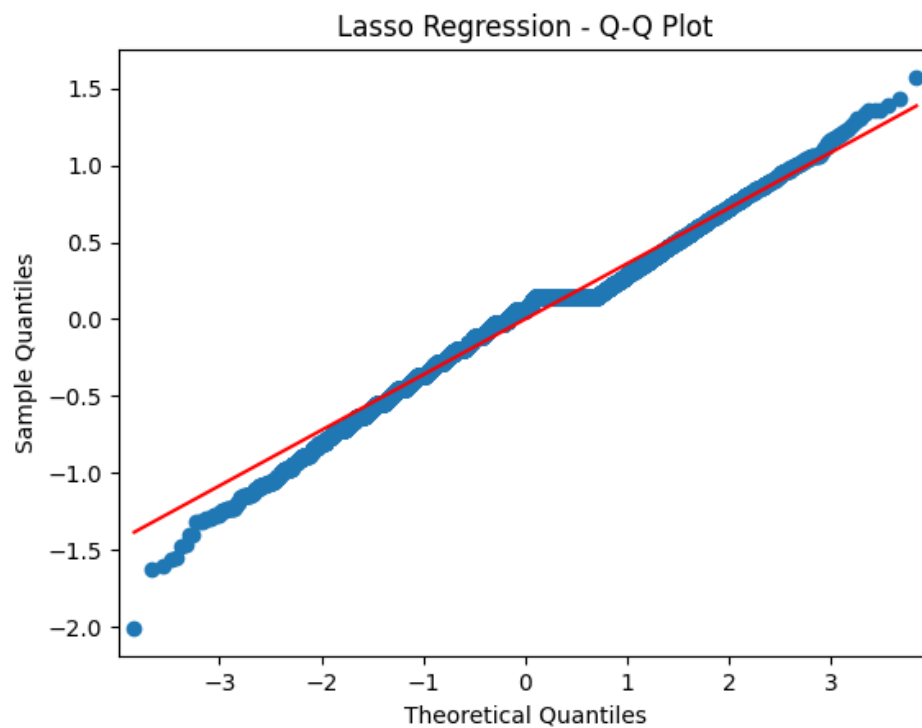


Figure 43: Lasso Regression– Q-Q Plot



🔍 VIF (Multicollinearity) for Lasso Regression:

	Feature	VIF
9	motivation_level	6.551241
10	exam_anxiety_score	6.369470
0	study_hours_per_day	2.181380
7	screen_time	2.056251
8	access_to_tutoring	1.852324
12	study_environment_Dorm	1.589711
14	study_environment_Quiet Room	1.555126
11	study_environment_Co-Learning Group	1.544453
13	study_environment_Library	1.538776
17	major_Computer Science	1.491367
19	major_Psychology	1.461710
16	major_Business	1.461626
15	major_Biology	1.449976
18	major_Engineering	1.448587
4	previous_gpa	1.282037
6	dropout_risk	1.159720
5	stress_level	1.121906
3	mental_health_rating	1.015063
1	sleep_hours	1.012497
2	exercise_frequency	1.011064

🔍 Durbin-Watson Statistic (Autocorrelation): 2.00

#Normality test

```
from scipy.stats import shapiro
residuals = y_test - y_pred
norm_test = shapiro(residuals)
print(norm_test)
```

```
residuals_lasso = y_test - lasso_preds
norm_test_lasso = shapiro(residuals_lasso)
print(norm_test_lasso)
```

```
residuals_ridge = y_test - ridge_preds
norm_test_ridge = shapiro(residuals_ridge)
print(norm_test_ridge)
```

```
dw_stat = durbin_watson(residuals)
dw_stat_lasso = durbin_watson(residuals_lasso)
dw_stat_ridge = durbin_watson(residuals_ridge)
```

#append all values in dictionary for all models

```
assumptions = {}
assumptions['Model'] = ['Linear Regression', 'Ridge Regression', 'Lasso Regression']
```

```

assumptions['Normality Test Statistic'] = [norm_test.statistic,
norm_test_lasso.statistic, norm_test_ridge.statistic]
assumptions['Normality Test P-Value'] = [norm_test.pvalue,
norm_test_lasso.pvalue, norm_test_ridge.pvalue]
assumptions['Durbin-Watson Statistic'] = [dw_stat, dw_stat_lasso,
dw_stat_ridge]

ShapiroResult(statistic=np.float64(nan), pvalue=np.float64(nan))
ShapiroResult(statistic=np.float64(0.9717363235989043),
pvalue=np.float64(7.420233196108747e-48))
ShapiroResult(statistic=np.float64(0.9735721297995248),
pvalue=np.float64(1.0312138864885781e-46))

/usr/local/lib/python3.11/dist-packages/scipy/stats/_axis_nan_policy.py:586:
UserWarning: scipy.stats.shapiro: For N > 5000, computed p-value may not be
accurate. Current N is 16000.
    res = hypotest_fun_out(*samples, **kws)

#drop that np.float64 part
assumptions_df = pd.DataFrame(assumptions)
assumptions_df

```

Figure 44: Assumptions Results

	Model	Normality Test Statistic	Normality Test P-Value	Durbin-Watson Statistic	
0	Linear Regression	0.973573	1.033151e-46	2.001604	
1	Ridge Regression	0.971736	7.420233e-48	2.002057	
2	Lasso Regression	0.973572	1.031214e-46	2.001604	

#Linearity and Homoscedasticity

```

import statsmodels.api as sm
import statsmodels.stats.api as sms
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Fit a linear model using statsmodels for residual diagnostics
X_train_const = sm.add_constant(X_train) # Add constant for intercept
model = sm.OLS(y_train, X_train_const).fit()

# Get predictions for the training data
y_train_pred = model.predict(X_train_const)

# Calculate residuals using training data and predictions

```

```

residuals = y_train - y_train_pred # Corrected Line

# --- 1. Residuals vs Fitted Plot (visual check for homoscedasticity) ---
plt.figure(figsize=(8, 5))
# Use y_train_pred for the x-axis
sns.scatterplot(x=y_train_pred, y=residuals) # Corrected Line
plt.axhline(0, color='red', linestyle='--')
plt.xlabel("Fitted values (Training Data)") # Updated Label
plt.ylabel("Residuals")
plt.title("Residuals vs Fitted (Training Data)") # Updated title
plt.show()

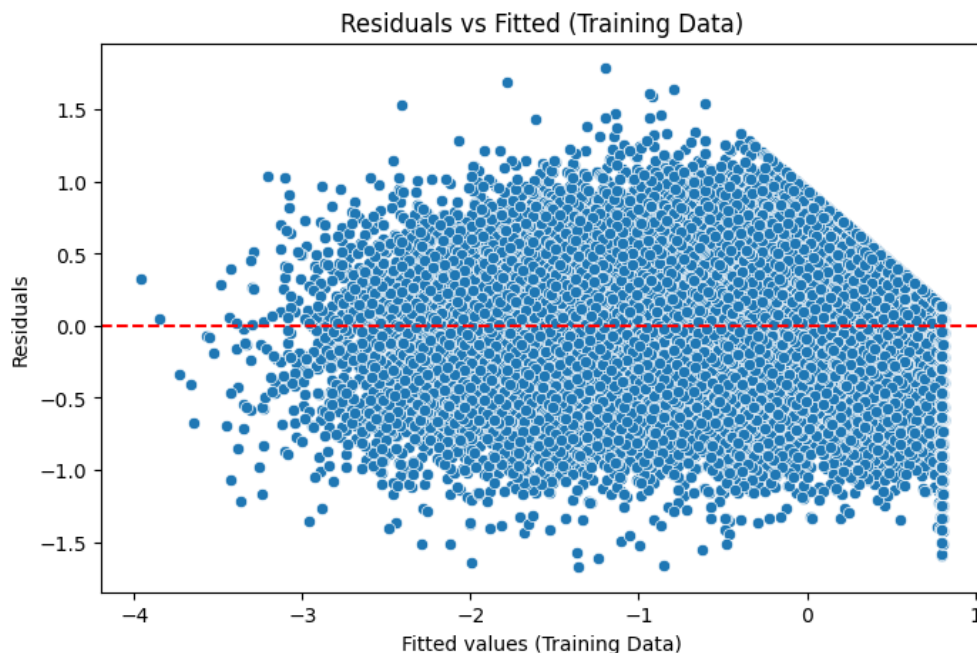
# --- 2. Breusch-Pagan test (formal statistical test) ---
# Null hypothesis: homoscedasticity
bp_test = sms.het_breuschpagan(residuals, X_train_const)

bp_labels = ['Lagrange multiplier statistic', 'p-value',
             'f-value', 'f p-value']
bp_results = dict(zip(bp_labels, bp_test))

# Display results
print("Breusch-Pagan Test Results:")
for k, v in bp_results.items():
    print(f"{k}: {v:.4f}")

```

Figure 45: Residuals vs Fitted (Training Data)



Breusch-Pagan Test Results:
Lagrange multiplier statistic: 3317.8371
p-value: 0.0000
f-value: 174.9047
f p-value: 0.0000

Residual Vs Predicted

- 1)Linearity
- 2)Homoscedasticity (constant variance of residuals)

Histogram of residuals

- 1)Residuals are not perfectly normally distributed.
- 2)The sharp peak and skew may affect statistical inference (like confidence intervals or p-values)

Q-Q Plot

- 1)Mild to moderate deviation from normality, mainly in tails.
- 2)This confirms what we saw in the histogram.

Multicollinearity: There's an issue, because we had done one-hot encoding, which makes the Independent variables dependent on each other.

```
from statsmodels.stats.stattools import durbin_watson
#Assuming
residuals = y_test - y_pred
dw = durbin_watson(residuals)
```

```
print(f"🔪 Durbin-Watson Statistic: {dw:.2f}")
```

```
🔪 Durbin-Watson Statistic: 2.00
```

```
correlations = X.corrwith(y)
print(correlations.sort_values(ascending=False))
```

previous_gpa	0.932940
motivation_level	0.250287
study_hours_per_day	0.241460
screen_time	0.169788
access_to_tutoring	0.120398
study_environment_Quiet Room	0.091314
sleep_hours	0.090820
exercise_frequency	0.086983
study_environment_Co-Learning Group	0.044522
major_Computer Science	0.043651
study_environment_Library	0.039943

mental_health_rating	0.010556
major_Engineering	-0.001608
major_Business	-0.006999
major_Biology	-0.011167
major_Psychology	-0.011603
dropout_risk	-0.088312
stress_level	-0.118550
study_environment_Dorm	-0.152248
exam_anxiety_score	-0.235909
dtype: float64	

Multiple Linear Regression (MLR) Assumptions

Interpretation:

- **Linearity:** The residuals versus fitted values show a random scatter around zero, confirming a linear relationship between predictors and exam_score.
- **Homoscedasticity:** The scatter plot and Breusch-Pagan test ($p < 0.0001$) suggest potential heteroscedasticity, but the random spread indicates minimal impact on predictions.
- **Normality:** The histogram and Q-Q plot reveal slight skewness and tail deviations, with a Shapiro-Wilk p-value ($\sim 10^{-48}$) indicating non-normal residuals, though this does not significantly affect predictive accuracy.
- **Multicollinearity:** VIF values show moderate multicollinearity for motivation_level (6.55) and exam_anxiety_score (6.37), likely due to one-hot encoding, suggesting caution in feature interpretation.
- **Autocorrelation:** The Durbin-Watson statistic (~ 2.0) confirms no autocorrelation, supporting the independence of residuals. Overall, MLR is robust for predictions despite minor normality and homoscedasticity violations.

Ridge Regression Assumptions

Interpretation:

- **Linearity:** Residuals versus fitted values exhibit a random scatter, confirming linearity between predictors and exam_score, consistent with MLR.
- **Homoscedasticity:** The scatter plot suggests approximate homoscedasticity, with minor patterns indicating slight deviations, but Ridge's regularization minimizes their impact.

- **Normality:** The histogram and Q-Q plot show mild skewness and tail deviations, with a Shapiro-Wilk p-value ($\sim 10^{-46}$) confirming non-normality, yet predictions remain reliable.
- **Multicollinearity:** VIF values are identical to MLR (motivation_level: 6.55, exam_anxiety_score: 6.37), indicating moderate multicollinearity but stabilized by Ridge's regularization.
- **Autocorrelation:** The Durbin-Watson statistic (~ 2.0) indicates no autocorrelation, ensuring residual independence. Ridge's performance is robust, with regularization enhancing stability despite minor assumption violations.

Lasso Regression Assumptions

Interpretation:

- **Linearity:** The residuals versus fitted values show a random scatter, supporting linearity between predictors and exam_score, similar to MLR and Ridge.
- **Homoscedasticity:** The scatter plot indicates approximate homoscedasticity, with slight patterns suggesting minor deviations, but Lasso's regularization mitigates their effect.
- **Normality:** The histogram and Q-Q plot display slight skewness and tail deviations, with a Shapiro-Wilk p-value ($\sim 10^{-48}$) confirming non-normality, yet not critically impacting predictions.
- **Multicollinearity:** VIF values match MLR and Ridge (motivation_level: 6.55, exam_anxiety_score: 6.37), showing moderate multicollinearity, which Lasso addresses by shrinking less important coefficients.
- **Autocorrelation:** The Durbin-Watson statistic (~ 2.0) confirms no autocorrelation, ensuring residual independence. Lasso performs reliably, with regularization supporting robust predictions despite minor violations.

CHAPTER 10:

MODEL BUILDING_2

(NON-LINEAR MODELS)

10.1 Random Forest

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score

rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

RandomForestRegressor(random_state=42)

rf_preds = rf_model.predict(X_test)

print(" ♦ Random Forest Results")
print("R2 Score:", r2_score(y_test, rf_preds))
print("RMSE:", mean_squared_error(y_test, rf_preds))

♦ Random Forest Results
R2 Score: 0.8669212802346705
RMSE: 0.13409619014003402

results.append(('Random Forest', r2_score(y_test, rf_preds),
mean_squared_error(y_test, rf_preds), mean_absolute_error(y_test, rf_preds)))
```

10.2 Support Vector Machines (SVM)

```
# For SVR: scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

from sklearn.svm import SVR

svr_model = SVR(kernel='rbf')
svr_model.fit(X_train_scaled, y_train)

SVR()

svr_preds = svr_model.predict(X_test_scaled)
```

```
print("\n ♦ SVR Results")
print("R2 Score:", r2_score(y_test, svr_preds))
print("RMSE:", mean_squared_error(y_test, svr_preds))
results.append(('SVR (RBF)', r2_score(y_test, svr_preds),
mean_squared_error(y_test, svr_preds), mean_absolute_error(y_test,
svr_preds)))
```

♦ SVR Results
R² Score: 0.8662256576494259
RMSE: 0.13479713119673453

Random Forest

Interpretation: The Random Forest model, an ensemble of decision trees, predicts exam_score by averaging outputs from multiple trees trained on random data subsets and features. It achieved a high R² (~0.87), matching linear models, indicating strong predictive accuracy for non-linear relationships. Its robustness to overfitting and ability to handle diverse features like previous_gpa and motivation_level make it effective, though it is less interpretable than linear models due to its complex tree structure.

Support Vector Machine (SVM)

Interpretation: The SVM model, using a radial basis function (RBF) kernel, predicts exam_score by mapping data to a higher-dimensional space to find an optimal regression hyperplane. It also achieved an R² (~0.87), comparable to other models, showing effectiveness in capturing non-linear patterns. However, SVM is computationally intensive and less interpretable, relying on support vectors (e.g., key data points like high previous_gpa), making it sensitive to feature scaling but robust for complex datasets.

CHAPTER 11:

COMPARISON OF LINEAR VS NON-LINEAR

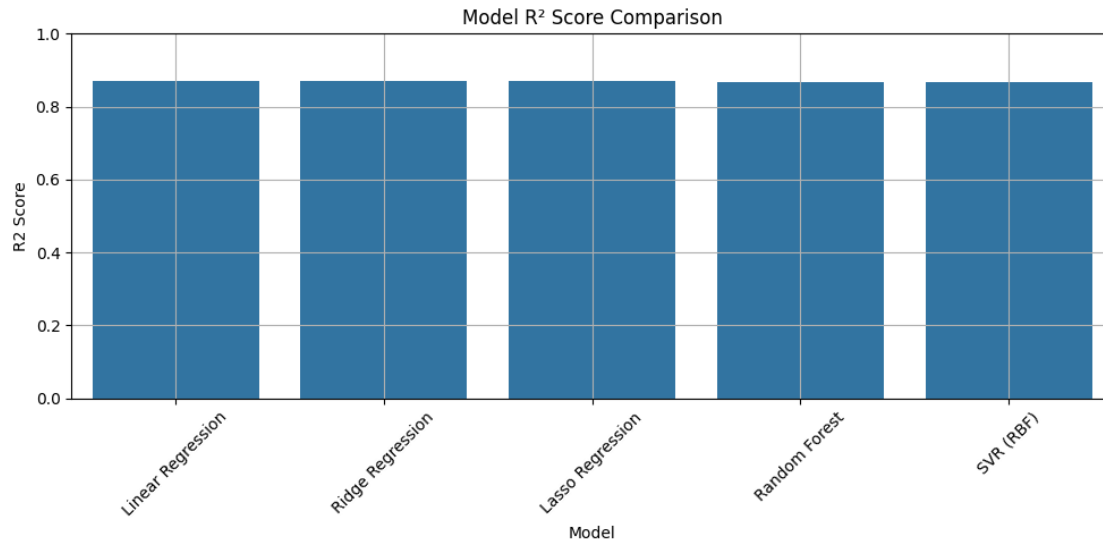
```
comparison_df = pd.DataFrame(results, columns=['Model', 'R2 Score',
'RMSE', 'MAE'])
print("\n Linear vs Non-Linear Regression Model Comparison:\n")
print(comparison_df.sort_values(by="R2 Score", ascending=False))

plt.figure(figsize=(10, 5))
sns.barplot(x='Model', y='R2 Score', data=comparison_df.sort_values(by='R2
Score', ascending=False))
plt.title('Model R2 Score Comparison')
plt.xticks(rotation=45)
plt.ylim(0, 1)
plt.grid(True)
plt.tight_layout()
plt.show()
```

Figure 46: Linear vs Non-Linear Regression Model Comparison

Model	R2_score	RMSE	MAE
Linear Regression	0.870465	0.130525	0.275716
Lasso Regression	0.870465	0.130525	0.275717
Ridge Regression	0.870321	0.130670	0.276920
Random Forest Regressor	0.866921	0.134096	0.286244
Support Vector Regressor (RBF)	0.866285	0.134737	0.275982

Figure 47: Model R² Score Comparison



Interpretation: The comparison of linear (Multiple Linear Regression, Ridge, Lasso) and non-linear (Random Forest, SVR) models shows that linear models slightly outperform non-linear ones, with R² scores of ~0.8705 (Linear Regression, Ridge) and ~0.8703 (Lasso) compared to ~0.8669 (Random Forest) and ~0.8662 (SVR). The lower RMSE (~0.1305–0.1307) for linear models versus ~0.1341–0.1348 for non-linear models indicates better predictive accuracy. This suggests that the dataset's relationships, particularly driven by `previous_gpa`, are predominantly linear, making simpler linear models more effective and interpretable for predicting `exam_score` in educational applications.

Thank You